AWS COST TUI: Linux-Based AWS Cost Monitoring & Optimization

PROBLEM

Whenever I wanted to check my AWS costs, I had to log in to the AWS Console and navigate to Cost Explorer.

I wanted a simple way to monitor daily AWS spending, EC2 instance costs, and overall cloud expenses directly from my Linux terminal, without repeatedly opening the AWS Console.

HOW I DEBUGGED IT

I wanted a quicker way to check AWS costs without logging in to the console every time. So, I explored using Python and the AWS Cost Explorer API to fetch cost data and display it directly in the Linux terminal.

THE SOLUTION **AWS Account → Cost Explorer API → Python TUI → Cost Dashboard**

- Built a Python-based AWS Cost TUI to view cloud costs directly from the terminal.Added daily spending, EC2 instance costs, and service-wise cost breakdowns.
- Created an HTML dashboard with cost trends and resource details.
- Added CSV exports and optional Slack/Teams reporting for easier monitoring.

NOTE: Complete source code, setup steps, and AWS cost verification screenshots are included below.

Has anyone else faced this problem?

How did you fix it?

Blog:https://vishnusai.hashnode.dev/aws-cost-tui-linux-based-aws-cost-monitoring-optimization

HashnodProfile:https://hashnode.com/@vishnusaiK

GitHub : https://github.com/ksaivishnusaikvs/DevOps-Engineering/tree/main/Linkedin/Post

```
ubuntu@ip-172-31-29-13:~$ ls
Audit.py            audit_report.pdf  bumblebee-deep.ndjson        bumblebee-inventory.ndjson      cost.py          go              v5cost.py
apps_data           audit_run.log     bumblebee-devops.ndjson      bumblebee-projects-root.ndjson  dashboard.html   three-tier-app  v6cost.py
audit.py            auditec2.py       bumblebee-ec2.ndjson         bumblebee-projects.ndjson       deleteapprove.py v3cost.py       watch_config.json
audit_report.docx   aws-ai            bumblebee-full-linux.ndjson  c2.py                           devops           v4cost.py
ubuntu@ip-172-31-29-13:~$ vi v7cost.py
ubuntu@ip-172-31-29-13:~$ vi v8cost.py
ubuntu@ip-172-31-29-13:~$ chmod +x v8cost.py
ubuntu@ip-172-31-29-13:~$ ./v8cost.py
2026-10-25 00:56:02,251 INFO Found credentials from IAM Role: adminrole
2026-10-25 00:56:03,642 WARNING Resource-level cost unavailable (enable resource-level data in Cost Explorer preferences): An error occurred (AccessDeniedException) when
 calling the GetCostAndUsageWithResources operation: Resource-level data granularity is an opt-in only feature. You can be enable this feature from the PAYER account's C
ost Explorer Settings page.
```

AWS COST REPORT · AWS · Sun 25 Oct 2026

YESTERDAY	LAST 7 DAYS	MONTH TO DATE	LAST MONTH	FORECAST
$1.56	$20.76	$564.48	$1,052.14	$1,150.17
Sat 24 Oct	-37% vs prev 7d	October 2026		+9% vs last month

DAILY COST

last 30d — min $1.56 avg $26.35 max $126.50

	MON	TUE	WED	THU	FRI	SAT	SUN
Week 1 24 Aug							$1.43
Week 2 31 Aug	$1.49	$162.15	$1.65	$1.59	$1.58	$1.45	$1.45
Week 3 07 Sep	$19.49	$36.42	$37.87	$38.92	$38.45	$37.05	$37.05
Week 4 14 Sep	$37.05	$37.05	$37.14	$37.08	$37.28	$37.58	$37.58
Week 5 21 Sep	$37.58	$37.58	$37.58	$37.58	$37.58	$37.58	$37.58
Week 6 28 Sep	$37.65	$37.66	$37.86	$126.50	$48.07	$48.85	$48.97
Week 7 05 Oct	$48.99	$47.48	$40.10	$34.64	$34.58	$32.34	$14.01
Week 8 12 Oct	$3.19	$3.17	$3.25	$3.21	$3.18	$3.18	$3.18
Week 9 19 Oct	$3.12	$3.10	$3.36	$3.49	$2.96	$1.56	

Color meaning: Low (< $3.18) · Medium ($3.18–$37.05) · High ($37.05–$37.65) · Very High (> $37.65)

SERVICE-WISE

#	Service	Last month	MTD	Last 7d	Yday	Now	Share
1	Relational Database Se	749.35	**318.22**	0.15	0.00	▼99%	56.4%
2	Tax	160.50	**86.11**	0.00	0.00	n/a	15.3%
3	Elastic Compute Cloud	3.62	**62.72**	0.47	0.00	▲148%	11.1%
4	EC2 - Other	35.94	**52.84**	14.50	1.05	▼10%	9.4%
5	Lightsail	13.72	**13.74**	3.87	0.39	▼5%	2.4%
6	Elastic Container Serv	38.75	**8.26**	0.00	0.00	n/a	1.5%
7	Elastic Load Balancing	25.31	**6.45**	0.00	0.00	n/a	1.1%
8	AmazonCloudWatch	14.12	**6.33**	0.02	0.00	▼94%	1.1%
9	Virtual Private Cloud	10.02	**5.96**	1.03	0.08	▲21%	1.1%
10	Cost Explorer	0.00	**1.57**	0.24	0.00	▼64%	0.3%
11	Key Management Service	0.00	**1.42**	0.43	0.04	▼3%	0.3%
12	Route 53	0.50	**0.50**	0.00	0.00	n/a	0.1%
	Other (2)	0.31	**0.36**	0.06	0.00	▼21%	0.1%
	TOTAL	1,052.14	**564.48**	20.76	1.56	▼37%	100.0%

CATEGORY-WISE

Database 56% · Compute 24% · Other 16% · Networking 2% · Management 1% · Security 0%

#	Category	Last month	MTD	Last 7d	Yday	Now	Share
1	Database	749.35	**318.22**	0.15	0.00	▼99%	56.4%
2	Compute	92.02	**137.56**	18.84	1.44	▼7%	24.4%
3	Other	160.50	**87.90**	0.30	0.00	▼59%	15.6%
4	Networking	35.83	**12.92**	1.03	0.08	▲21%	2.3%
5	Management	14.12	**6.33**	0.02	0.00	▼94%	1.1%
6	Security	0.31	**1.55**	0.43	0.04	▼5%	0.3%
	TOTAL	1,052.14	**564.48**	20.76	1.56	▼37%	100.0%

REGION-WISE + EC2 FLEET

#	Region	MTD	Yday	Now	Run	Stop	EBS GB	Idle*
1	us-east-2	**385.56**	0.00	▼77%	-	-	-	-
2	NoRegion	**86.11**	0.00	n/a	-	-	-	-
3	ap-south-1	**59.56**	1.36	▼5%	0	0	0	0
4	us-west-2	**19.51**	0.18	▼14%	1	0	120	0
5	us-east-1	**13.03**	0.02	▲24%	-	-	-	-
6	global	**0.72**	0.00	▼14%	-	-	-	-

* idle = unattached EBS volumes + unassociated Elastic IPs (~$3.65/mo each EIP)

EC2 FLEET

1 running **0** stopped **1** regions **0** unattached volumes (0 GB) **0** idle Elastic IPs

EC2 COST BY INSTANCE TYPE (last 30d)

Instance type	Region	Cost 30d	Running
t2.2xlarge	us-east-2	**50.58**	-
t2.large	us-west-2	**11.33**	1
t3.micro	us-east-1	**1.19**	-

ALL SERVICES (active / idle)

#	Service	Status	MTD	Last 7d	Yday	Last billed
1	Relational Database Service	○ IDLE	**318.22**	0.15	0.00	23 Oct
2	Tax	○ IDLE	**86.11**	0.00	0.00	01 Oct
3	Elastic Compute Cloud - Compute	○ IDLE	**62.72**	0.47	0.00	23 Oct
4	EC2 - Other	● ACTIVE	**52.84**	14.50	1.05	24 Oct
5	Lightsail	● ACTIVE	**13.74**	3.87	0.39	24 Oct
6	Elastic Container Service	○ IDLE	**8.26**	0.00	0.00	07 Oct
7	Elastic Load Balancing	○ IDLE	**6.45**	0.00	0.00	06 Oct
8	AmazonCloudWatch	○ IDLE	**6.33**	0.02	0.00	22 Oct
9	Virtual Private Cloud	● ACTIVE	**5.96**	1.03	0.08	24 Oct
10	Cost Explorer	○ IDLE	**1.57**	0.24	0.00	23 Oct
11	Key Management Service	● ACTIVE	**1.42**	0.43	0.04	24 Oct
12	Route 53	○ IDLE	**0.50**	0.00	0.00	01 Oct
13	Budgets	○ IDLE	**0.22**	0.06	0.00	23 Oct
14	Secrets Manager	○ IDLE	**0.14**	0.00	0.00	11 Oct

4 active **10** idle **14** services (active = billed yesterday)

ALL EC2 INSTANCES (running / stopped)

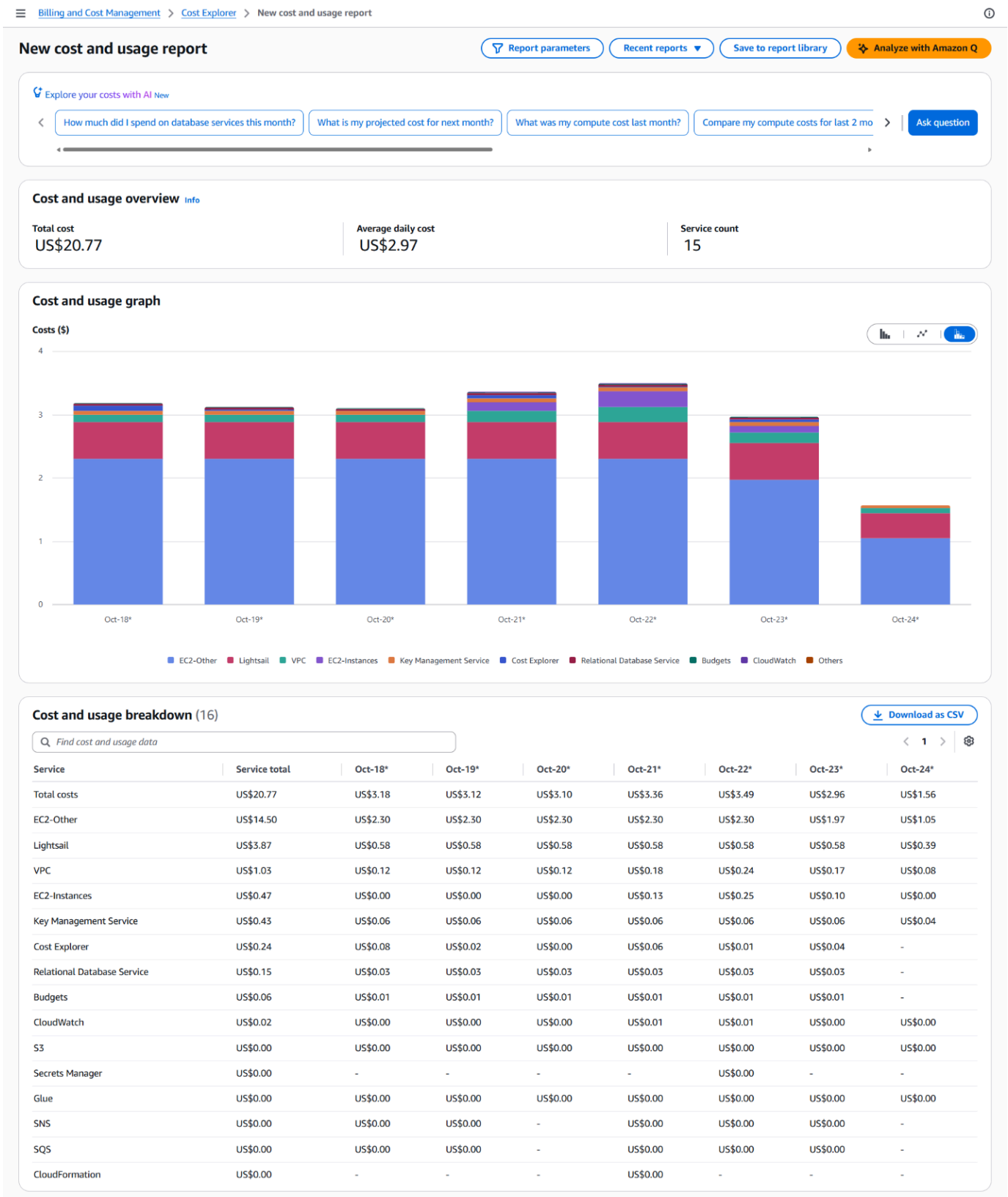
#	Name	Instance ID	Type	Region	State	Cost 13d
1	mlop	i-05fca5e7d3cef6377	t2.large	us-west-2	● running	-

1 running **0** stopped **0** other

Cost Explorer, unblended cost; credits/refunds excluded unless --include-credits

```
ubuntu@ip-172-31-29-13:~$
```

Step1: Last 7days and daily cost check verification the values are matching



Cost and usage breakdown (16)

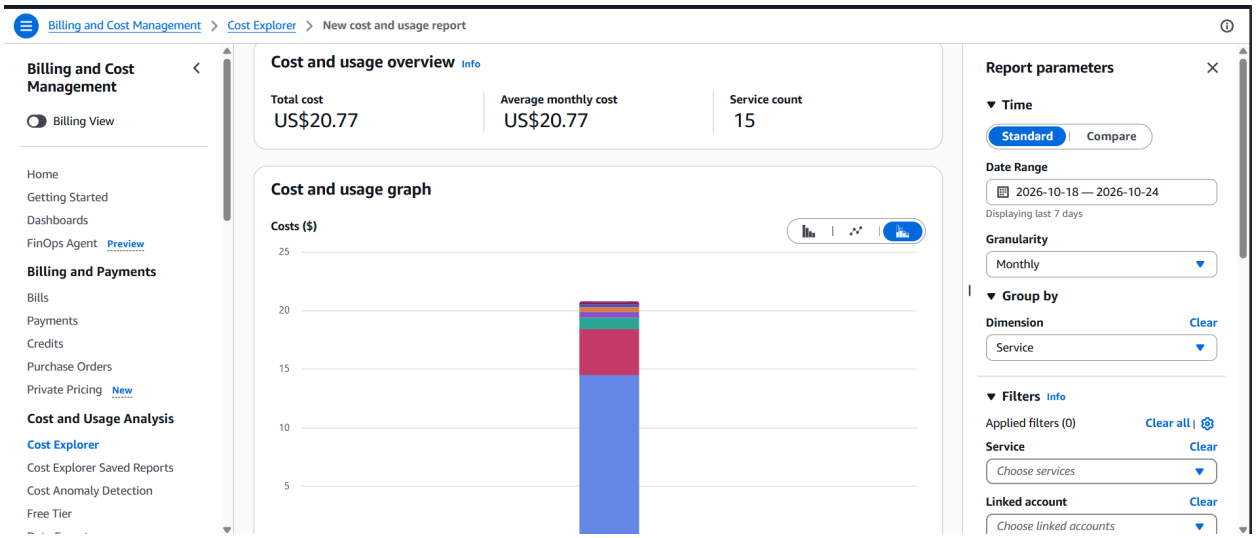
Download as CSV

Find cost and usage data

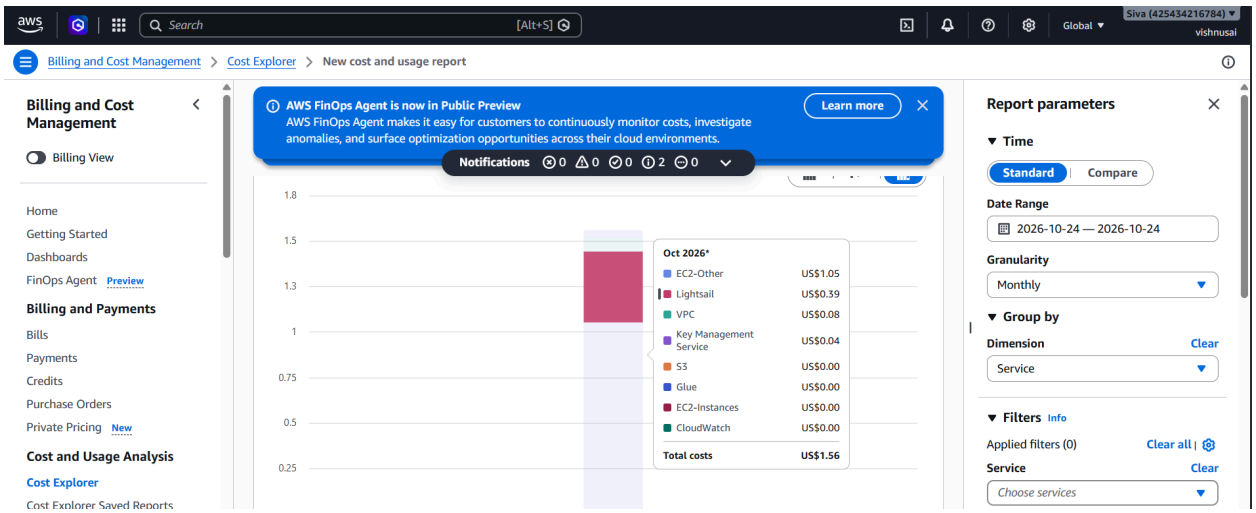
1

Service	Service total	Oct-18*	Oct-19*	Oct-20*	Oct-21*	Oct-22*	Oct-23*	Oct-24*
Total costs	US$20.77	US$3.18	US$3.12	US$3.10	US$3.36	US$3.49	US$2.96	US$1.56
EC2-Other	US$14.50	US$2.30	US$2.30	US$2.30	US$2.30	US$2.30	US$1.97	US$1.05
Lightsail	US$3.87	US$0.58	US$0.58	US$0.58	US$0.58	US$0.58	US$0.58	US$0.39
VPC	US$1.03	US$0.12	US$0.12	US$0.12	US$0.18	US$0.24	US$0.17	US$0.08
EC2-Instances	US$0.47	US$0.00	US$0.00	US$0.00	US$0.13	US$0.25	US$0.10	US$0.00
Key Management Service	US$0.43	US$0.06	US$0.06	US$0.06	US$0.06	US$0.06	US$0.06	US$0.04
Cost Explorer	US$0.24	US$0.08	US$0.02	US$0.00	US$0.06	US$0.01	US$0.04	-
Relational Database Service	US$0.15	US$0.03	US$0.03	US$0.03	US$0.03	US$0.03	US$0.03	-
Budgets	US$0.06	US$0.01	US$0.01	US$0.01	US$0.01	US$0.01	US$0.01	-
CloudWatch	US$0.02	US$0.00	US$0.00	US$0.00	US$0.01	US$0.01	US$0.00	US$0.00
S3	US$0.00	US$0.00	US$0.00	US$0.00	US$0.00	US$0.00	US$0.00	US$0.00
Secrets Manager	US$0.00	-	-	-	-	US$0.00	-	-
Glue	US$0.00	US$0.00	US$0.00	US$0.00	US$0.00	US$0.00	US$0.00	US$0.00
SNS	US$0.00	US$0.00	US$0.00	-	US$0.00	US$0.00	US$0.00	-
SQS	US$0.00	US$0.00	US$0.00	-	US$0.00	US$0.00	US$0.00	-
CloudFormation	US$0.00	-	-	-	US$0.00	-	-	-

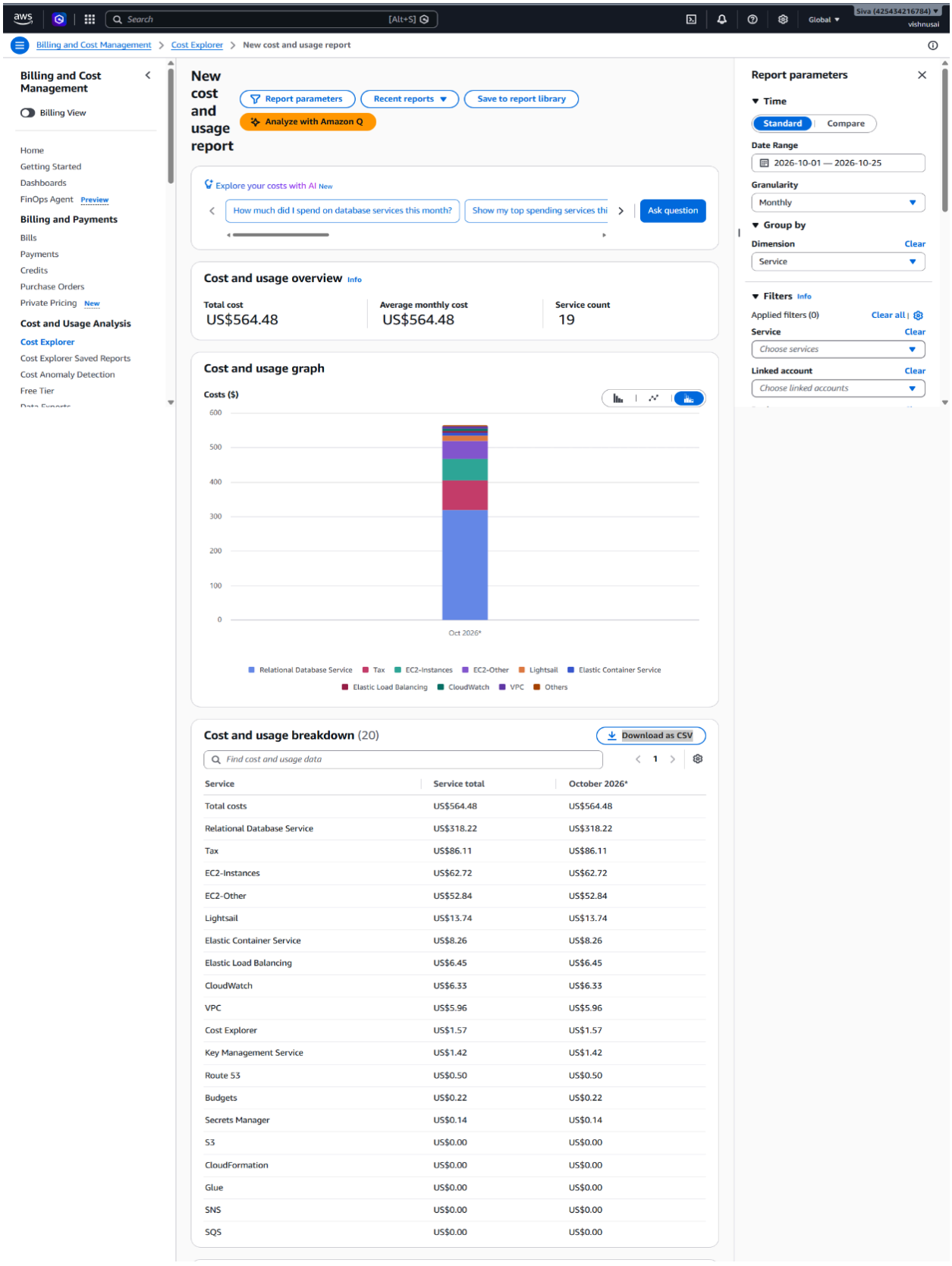
Step2: Last 7 days cost



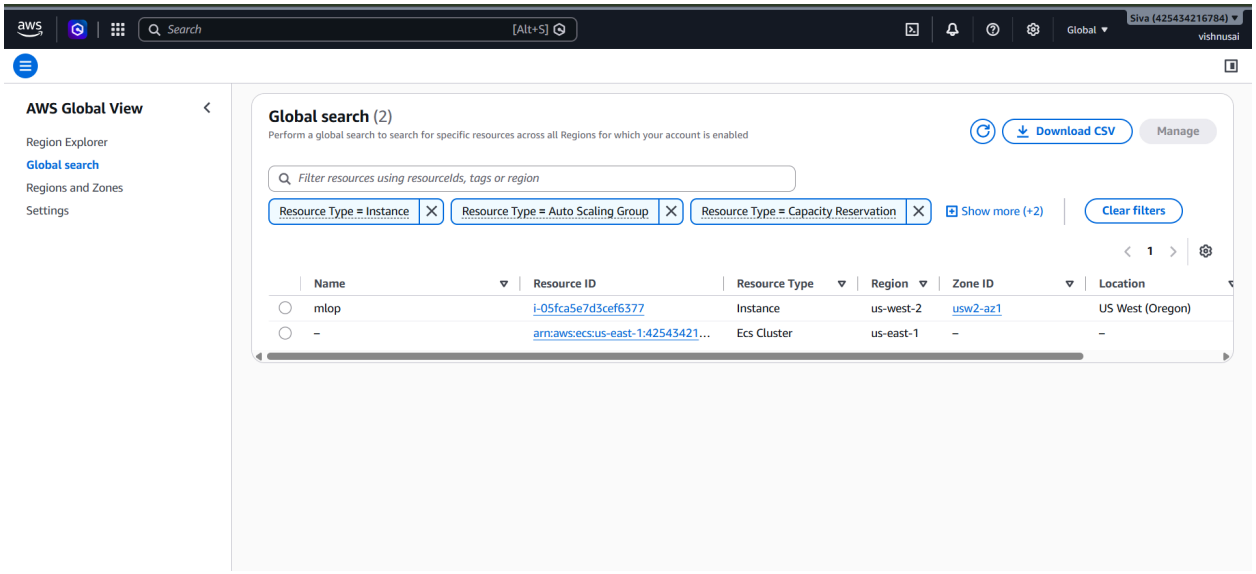
Step3: Last 1 day cost



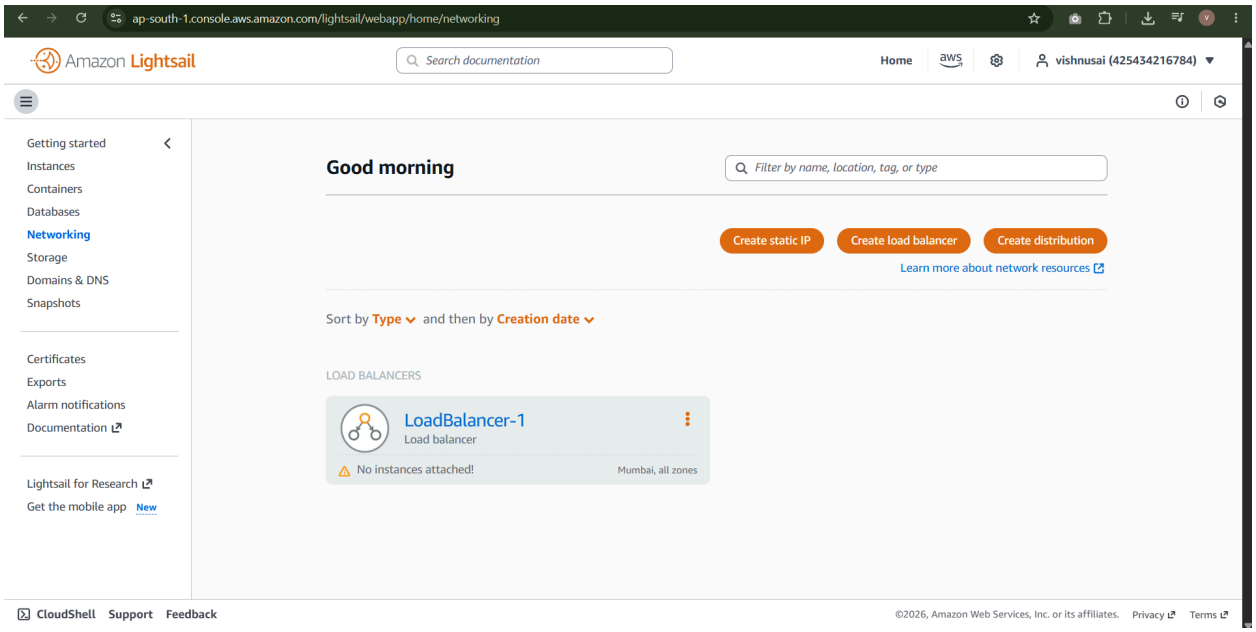
Step4: Last month to days cost



Step5: Resource Confirmation single ec2 is running see the above screen tui and check ecs cost also

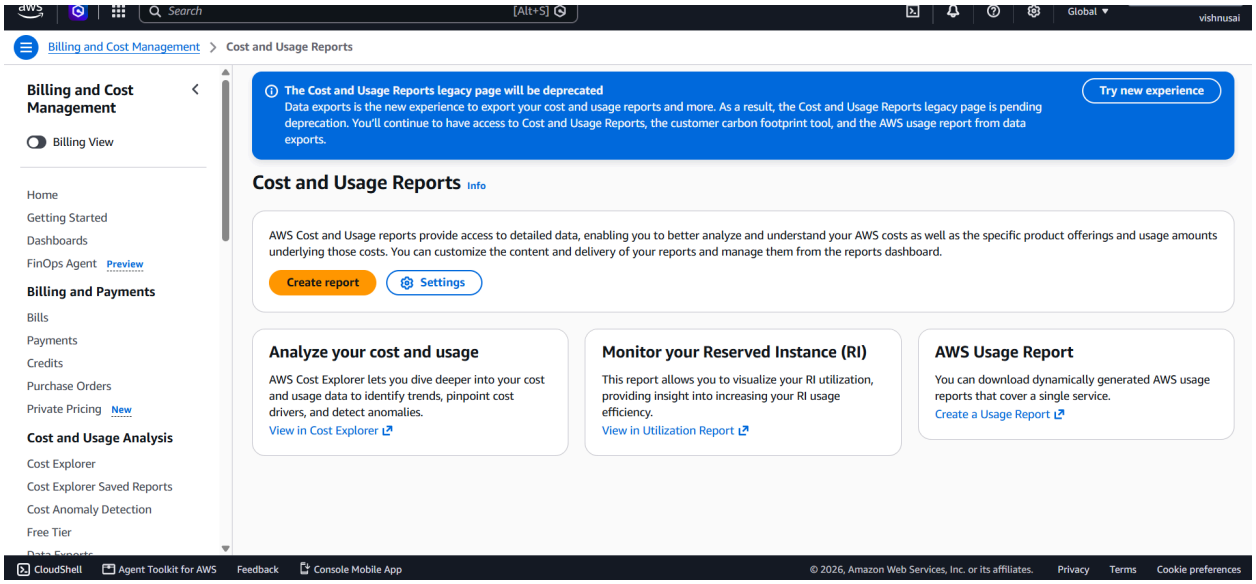


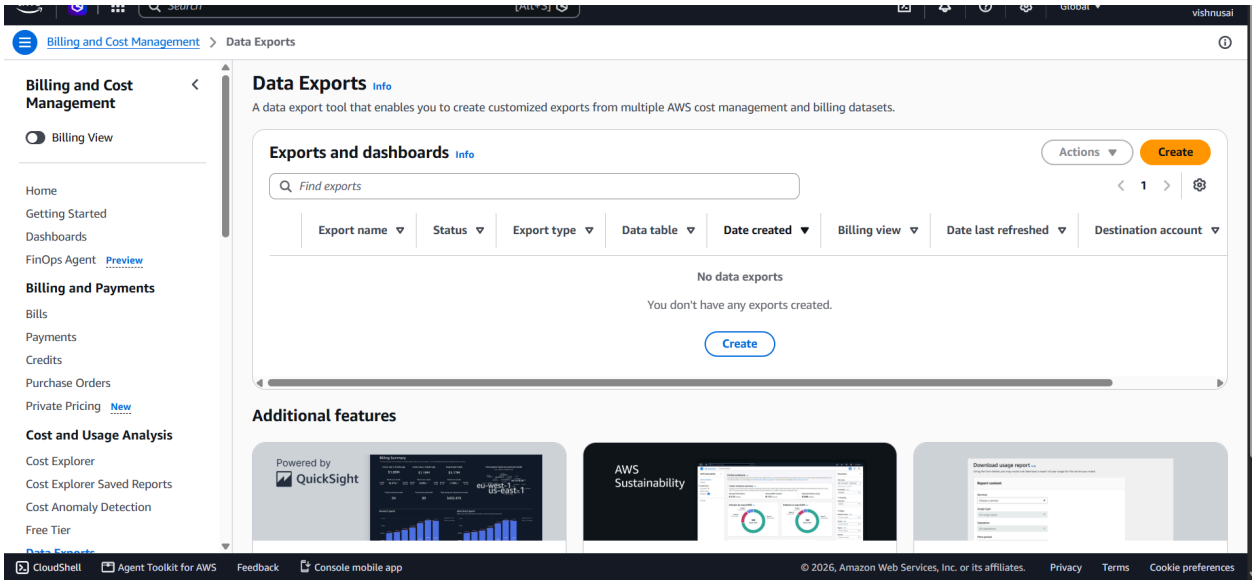
Step6: Check the resource is existing or not



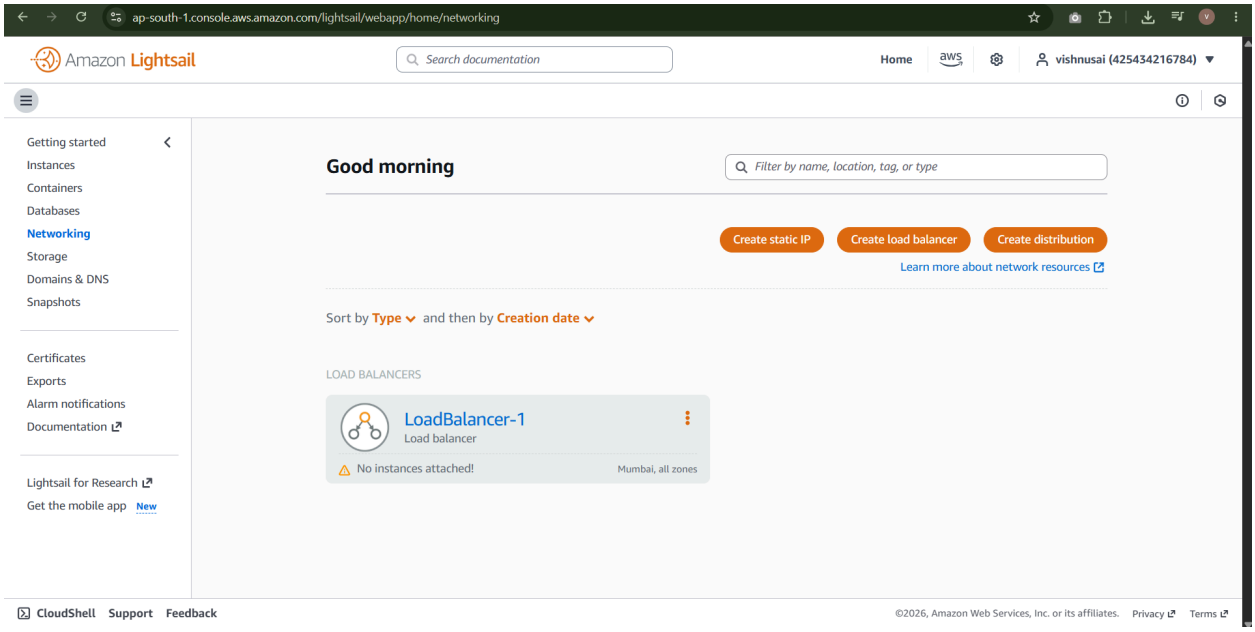
Step7: Yes. You can get AWS cost data directly through the AWS Cost Explorer API without creating a Cost and Usage Report (CUR).

You only need AWS credentials with the required Cost Explorer permissions. Your Python application can call the API and generate cost reports automatically.





Step8:hear is some ideal services that are not attached the check below code out it show idel




```
ALL SERVICES  (active / idle)
 #  Service                          Status          MTD      Last 7d      Yday  Last billed
 1  Relational Database Service      ○ IDLE       318.27         0.15      0.00       25 Oct
 2  Tax                              ○ IDLE        87.35         0.00      0.00       01 Oct
 3  Elastic Compute Cloud - Compute  ● ACTIVE      65.08         2.84      1.86       26 Oct
 4  EC2 - Other                      ● ACTIVE      56.17        13.24      0.90       26 Oct
 5  Lightsail                        ○ IDLE        13.99         2.95      0.00       25 Oct
 6  Elastic Container Service        ○ IDLE         8.26         0.00      0.00       07 Oct
 7  AmazonCloudWatch                 ● ACTIVE       6.57         0.26      0.11       26 Oct
 8  Elastic Load Balancing           ○ IDLE         6.45         0.00      0.00       06 Oct
 9  Virtual Private Cloud            ● ACTIVE       6.28         1.10      0.13       26 Oct
10  Cost Explorer                    ● ACTIVE       1.79         0.35      0.04       26 Oct
11  Key Management Service           ● ACTIVE       1.54         0.42      0.03       26 Oct
12  Route 53                         ○ IDLE         0.50         0.00      0.00       01 Oct
13  Budgets                          ○ IDLE         0.24         0.06      0.00       25 Oct
14  Secrets Manager                  ○ IDLE         0.14         0.00      0.00       11 Oct
15  Simple Storage Service           ○ IDLE         0.01         0.00      0.00            -

   6 active   9 idle   15 services   (active = billed yesterday)

ALL EC2 INSTANCES  (running / stopped)
 #  Name                 Instance ID          Type        Region       State       Cost 13d
 1  mlop                 i-05fca5e7d3cef6377  t2.xlarge   us-west-2    ● running         -

   1 running   0 stopped   0 other

  Cost Explorer, unblended cost; credits/refunds excluded unless --include-credits
ubuntu@ip-172-31-29-13:~$
```

Model 3 : calendar cost dynamic

```
#!/usr/bin/env python3
"""
aws_cost_report.py - AWS cost report with a rich Linux terminal view and a self-contained
HTML dashboard: service / category / region wise cost, daily cost calendar, EC2 fleet
(per region, instance types, running/stopped), resource-level cost, idle-resource findings.
Output: console / CSV / HTML / Slack / Teams.

One file, no external config. Only dependency: boto3.

Cost: Cost Explorer ~$0.01 per API request. A run makes about 4 requests:
  1. get_cost_and_usage (SERVICE + REGION, daily; paginated)      - always
  2. get_cost_forecast                                            - always
  3. get_cost_and_usage (EC2 by INSTANCE_TYPE + REGION)           - skip with --no-ec2
  4. get_cost_and_usage_with_resources (EC2 resources, last 13d)  - skip with --no-resources
EC2 inventory (instances / EBS / Elastic IPs per region) uses the free EC2 Describe APIs
- skip with --no-inventory. Everything else is computed locally. Run it once a day.

Usage
-----
    python aws_cost_report.py --dry-run
    python aws_cost_report.py --html dashboard.html            # dashboard + console
    python aws_cost_report.py --html dashboard.html --days 180 # longer calendar history
    python aws_cost_report.py --slack --teams --html dashboard.html --csv cost.csv
    python aws_cost_report.py --tag Environment=production --label "AWS prod"
    python aws_cost_report.py --color never > report.txt       # plain text, no ANSI

Run every day (cron, regenerates the dashboard + posts to chat):
    0 8 * * * SLACK_WEBHOOK_URL=... /usr/bin/python3 /home/ubuntu/cost.py --html
/var/www/html/cost.html

Optional env: SLACK_WEBHOOK_URL, TEAMS_WEBHOOK_URL, COST_REPORT_LABEL,
COST_DASHBOARD_URL, NO_COLOR

IAM (read-only):
    ce:GetCostAndUsage, ce:GetCostAndUsageWithResources, ce:GetCostForecast       on "*"
    ssm:GetParameter (+ kms:Decrypt)  only if the webhook is read from SSM Parameter Store

Production: use an IAM role (no static keys); keep webhooks in SSM SecureString, not on the
command line
(--slack-webhook is visible in `ps`):
    aws ssm put-parameter --name /finops/slack-webhook --type SecureString --value
'https://hooks.slack.com/...'
```

```
    python3 cost.py --slack --slack-webhook-ssm /finops/slack-webhook --ssm-region ap-south-1
--html dash.html
Exit codes: 0 ok, 1 report/delivery failed (cron/CI can alert on it), 2 bad usage/config.
    ec2:DescribeRegions, ec2:DescribeInstances, ec2:DescribeVolumes,
ec2:DescribeAddresses

Resource-level cost needs "Resource-level data at daily granularity" enabled once in
Billing > Cost Explorer > Preferences (AWS only keeps ~14 days of it). If it is not
enabled the script just skips that section and tells you in the log.
"""
from __future__ import annotations

import argparse
import csv
import json
import logging
import os
import shutil
import sys
import time
import urllib.error
import urllib.request
from collections import defaultdict
from concurrent.futures import ThreadPoolExecutor, as_completed
from dataclasses import dataclass, field
from datetime import date, datetime, timedelta, timezone
from typing import Callable, Dict, List, Optional, Tuple

log = logging.getLogger("aws_cost_report")

EC2_COMPUTE = "Amazon Elastic Compute Cloud - Compute"
EC2_OTHER = "EC2 - Other"
RESOURCE_DAYS = 13          # Cost Explorer keeps resource-level data for ~14 days
EC2_TYPE_DAYS = 30


# ---------------------------------------------------------------------------
# Data model
# ---------------------------------------------------------------------------
@dataclass
class Row:
    service: str  # service / category / region name
    last_month: float = 0.0
    mtd: float = 0.0
```

```
    last7: float = 0.0
    prev7: float = 0.0
    yesterday: float = 0.0

    @property
    def wow_pct(self) -> Optional[float]:
        if self.prev7 <= 0:
            return None
        return (self.last7 - self.prev7) / self.prev7 * 100.0


@dataclass
class Anomaly:
    service: str
    yesterday: float
    baseline: float

    @property
    def delta(self) -> float:
        return self.yesterday - self.baseline


@dataclass
class Report:
    label: str
    run_date: date
    unit: str
    rows: List[Row]            # per service
    region_rows: List[Row]     # per region
    total: Row
    forecast_month_end: Optional[float]
    anomalies: List[Anomaly]
    dates: List[date]          # contiguous daily axis (start .. yesterday)
    daily: List[float]         # total cost per day (aligned with dates)
    series: List[dict]         # [{"s": service, "r": region, "k": category, "c": [daily costs]}]
    cat_rows: List[Row] = field(default_factory=list)       # per category
    ec2_types: List[dict] = field(default_factory=list)     # [{"r","t","c"}] last EC2_TYPE_DAYS
    ec2_days: int = EC2_TYPE_DAYS
    resources: List[dict] = field(default_factory=list)     # [{"id","k","r","c","n"}] top resources
    res_kinds: List[dict] = field(default_factory=list)     # [{"k","c","cnt"}]
    res_days: int = RESOURCE_DAYS
    inventory: List[dict] = field(default_factory=list)     # per-region EC2/EBS/EIP counts
    instances: List[dict] = field(default_factory=list)     # [{"id","n","t","st","r","c"}]
```

```
# ---------------------------------------------------------------------------
# Helpers
# ---------------------------------------------------------------------------
def money(x: float, unit: str = "USD") -> str:
    sym = "$" if unit == "USD" else ""
    suffix = "" if unit == "USD" else f" {unit}"
    return f"{sym}{x:,.2f}{suffix}"


def short_name(service: str) -> str:
    for prefix in ("Amazon ", "AWS "):
        if service.startswith(prefix):
            return service[len(prefix):]
    return service


CATEGORY_RULES: List[Tuple[str, Tuple[str, ...]]] = [
    ("Networking", ("cloudfront", "virtual private cloud", "vpc", "route 53", "load balanc", "api
gateway",
                    "direct connect", "global accelerator", "transit", "data transfer", "privatelink",
                    "network firewall", "vpn")),
    ("Compute", ("elastic compute cloud", "ec2", "lambda", "container", "kubernetes", "fargate",
"batch",
                 "lightsail", "beanstalk", "app runner", "autoscaling")),
    ("Storage", ("simple storage", "s3", "elastic file", "efs", "fsx", "backup", "glacier", "storage
gateway",
                 "ebs", "datasync")),
    ("Database", ("relational database", "rds", "dynamodb", "elasticache", "redshift",
"documentdb", "neptune",
                  "memorydb", "aurora", "timestream", "keyspaces")),
    ("Analytics", ("athena", "glue", "emr", "kinesis", "opensearch", "elasticsearch", "quicksight",
                   "managed streaming", "data pipeline", "lake formation", "msk")),
    ("AI / ML", ("sagemaker", "bedrock", "rekognition", "comprehend", "textract", "transcribe",
"polly",
                 "translate", "lex", "forecast", "personalize")),
    ("Security", ("key management", "secrets manager", "waf", "shield", "guardduty", "inspector",
                  "security hub", "firewall manager", "certificate", "cognito", "macie", "detective",
                  "iam", "security lake")),
    ("Management", ("cloudwatch", "cloudtrail", "config", "systems manager", "cloudformation",
"support",
                    "organizations", "x-ray", "control tower", "service catalog", "trusted advisor")),
    ("Integration", ("simple queue", "sqs", "notification", "sns", "eventbridge", "step functions",
                     "mq", "appsync", "simple email", "ses", "appflow")),
```

```
]


def categorize(service: str) -> str:
    s = service.lower()
    for cat, keys in CATEGORY_RULES:
        if any(k in s for k in keys):
            return cat
    return "Other"


def day_range(start: date, end_inclusive: date):
    n = (end_inclusive - start).days + 1
    for i in range(max(n, 0)):
        yield start + timedelta(days=i)


def period_sum(days: Dict[date, float], start: date, end_inclusive: date) -> float:
    return sum(days.get(d, 0.0) for d in day_range(start, end_inclusive))


def resource_kind(rid: str) -> str:
    if not rid or rid in ("NoResourceId", "No Resource Id"):
        return "Unallocated"
    tail = rid.rsplit("/", 1)[-1]
    for prefix, kind in (("i-", "EC2 instance"), ("vol-", "EBS volume"), ("snap-", "EBS snapshot"),
                         ("nat-", "NAT gateway"), ("eni-", "Network interface"), ("eipalloc-", "Elastic IP"),
                         ("vpce-", "VPC endpoint"), ("ami-", "AMI")):
        if tail.startswith(prefix):
            return kind
    if rid.startswith("arn:") and rid.count(":") >= 2:
        return rid.split(":")[2]
    return "Other"


# ---------------------------------------------------------------------------
# AWS Cost Explorer + inventory
# ---------------------------------------------------------------------------
def make_session(profile: Optional[str]):
    import boto3

    return boto3.Session(profile_name=profile) if profile else boto3.Session()
```

```
def make_ce_client(session):
    from botocore.config import Config

    # Cost Explorer is a global API served from us-east-1. Timeouts stop a stalled call from
hanging a cron job.
    cfg = Config(region_name="us-east-1", retries={"max_attempts": 8, "mode": "adaptive"},
                 connect_timeout=10, read_timeout=60)
    return session.client("ce", config=cfg)


def resolve_webhook(direct: Optional[str], ssm_name: Optional[str], session, ssm_region:
Optional[str],
                    name: str) -> str:
    """Webhook URL from (1) the value given directly / env var, else (2) an SSM Parameter Store
SecureString.
    Always HTTPS-only. The URL is a secret: it is never logged."""
    url = direct
    if not url and ssm_name:
        if session is None:
            session = make_session(None)
        region = ssm_region or session.region_name or os.getenv("AWS_REGION") or
os.getenv("AWS_DEFAULT_REGION")
        if not region:
            raise ValueError("SSM region unknown: pass --ssm-region or set AWS_REGION")
        resp = session.client("ssm", region_name=region).get_parameter(Name=ssm_name,
WithDecryption=True)
        url = resp["Parameter"]["Value"]
    url = (url or "").strip()
    if not url.lower().startswith("https://"):
        raise ValueError(f"{name} webhook must be an https:// URL")
    return url


def build_filter(include_credits: bool, tag: Optional[str]) -> Optional[dict]:
    parts = []
    if not include_credits:
        parts.append({"Not": {"Dimensions": {"Key": "RECORD_TYPE", "Values": ["Credit",
"Refund"]}}})
    if tag:
        key, _, value = tag.partition("=")
        if not key or not value:
            raise ValueError("--tag must look like KEY=VALUE")
        parts.append({"Tags": {"Key": key, "Values": [value]}})
    if not parts:
```

```
        return None
    return parts[0] if len(parts) == 1 else {"And": parts}


def and_filter(extra: dict, flt: Optional[dict]) -> dict:
    return extra if not flt else {"And": [extra, flt]}


def fetch_daily_costs(ce, start: date, end_exclusive: date, flt: Optional[dict]):
    """One paginated call -> {(service, region): {date: cost}}, currency unit."""
    costs: Dict[Tuple[str, str], Dict[date, float]] = defaultdict(lambda: defaultdict(float))
    unit = "USD"
    token = None
    while True:
        kwargs = dict(
            TimePeriod={"Start": start.isoformat(), "End": end_exclusive.isoformat()},
            Granularity="DAILY",
            Metrics=["UnblendedCost"],
            GroupBy=[{"Type": "DIMENSION", "Key": "SERVICE"},
                     {"Type": "DIMENSION", "Key": "REGION"}],
        )
        if flt:
            kwargs["Filter"] = flt
        if token:
            kwargs["NextPageToken"] = token
        resp = ce.get_cost_and_usage(**kwargs)
        for result in resp["ResultsByTime"]:
            d = date.fromisoformat(result["TimePeriod"]["Start"])
            for group in result["Groups"]:
                metric = group["Metrics"]["UnblendedCost"]
                svc, region = group["Keys"][0], group["Keys"][1]
                costs[(short_name(svc), region or "NoRegion")][d] += float(metric["Amount"])
                unit = metric.get("Unit", unit)
        token = resp.get("NextPageToken")
        if not token:
            break
    return costs, unit


def fetch_forecast(ce, today: date, flt: Optional[dict]) -> Optional[float]:
    from botocore.exceptions import BotoCoreError, ClientError

    next_month = (today.replace(day=1) + timedelta(days=32)).replace(day=1)
    try:
```

```
        kwargs = dict(
            TimePeriod={"Start": today.isoformat(), "End": next_month.isoformat()},
            Metric="UNBLENDED_COST",
            Granularity="MONTHLY",
        )
        if flt:
            kwargs["Filter"] = flt
        resp = ce.get_cost_forecast(**kwargs)
        return float(resp["Total"]["Amount"])
    except (ClientError, BotoCoreError, KeyError, ValueError) as exc:
        log.warning("Forecast unavailable: %s", exc)
        return None


def fetch_ec2_types(ce, today: date, flt: Optional[dict]) -> List[dict]:
    """EC2 compute cost by (region, instance type) for the last EC2_TYPE_DAYS days."""
    start = today - timedelta(days=EC2_TYPE_DAYS)
    f = and_filter({"Dimensions": {"Key": "SERVICE", "Values": [EC2_COMPUTE]}}, flt)
    agg: Dict[Tuple[str, str], float] = defaultdict(float)
    token = None
    while True:
        kwargs = dict(
            TimePeriod={"Start": start.isoformat(), "End": today.isoformat()},
            Granularity="MONTHLY", Metrics=["UnblendedCost"], Filter=f,
            GroupBy=[{"Type": "DIMENSION", "Key": "INSTANCE_TYPE"},
                     {"Type": "DIMENSION", "Key": "REGION"}],
        )
        if token:
            kwargs["NextPageToken"] = token
        resp = ce.get_cost_and_usage(**kwargs)
        for result in resp["ResultsByTime"]:
            for g in result["Groups"]:
                itype, region = g["Keys"][0], g["Keys"][1]
                agg[(region or "NoRegion", itype or "other")] +=
float(g["Metrics"]["UnblendedCost"]["Amount"])
        token = resp.get("NextPageToken")
        if not token:
            break
    out = [{"r": r, "t": t, "c": round(c, 2)} for (r, t), c in agg.items() if c >= 0.005]
    out.sort(key=lambda e: e["c"], reverse=True)
    return out


def fetch_resource_costs(ce, today: date, flt: Optional[dict]) -> Dict[str, dict]:
```

```
    """Per-resource cost (EC2 instances, EBS, NAT, ENI ...) for the last RESOURCE_DAYS
days."""
    start = today - timedelta(days=RESOURCE_DAYS)
    f = and_filter({"Dimensions": {"Key": "SERVICE", "Values": [EC2_COMPUTE, EC2_OTHER]}},
flt)
    res: Dict[str, dict] = {}
    token = None
    while True:
        kwargs = dict(
            TimePeriod={"Start": start.isoformat(), "End": today.isoformat()},
            Granularity="MONTHLY", Metrics=["UnblendedCost"], Filter=f,
            GroupBy=[{"Type": "DIMENSION", "Key": "RESOURCE_ID"},
                     {"Type": "DIMENSION", "Key": "REGION"}],
        )
        if token:
            kwargs["NextPageToken"] = token
        resp = ce.get_cost_and_usage_with_resources(**kwargs)
        for result in resp["ResultsByTime"]:
            for g in result["Groups"]:
                rid, region = g["Keys"][0], g["Keys"][1]
                e = res.setdefault(rid, {"id": rid, "r": region or "NoRegion", "c": 0.0})
                e["c"] += float(g["Metrics"]["UnblendedCost"]["Amount"])
        token = resp.get("NextPageToken")
        if not token:
            break
    return res


def fetch_inventory(session, workers: int = 8) -> Tuple[List[dict], List[dict]]:
    """Free EC2 Describe* calls in every enabled region -> (per-region summary, instances)."""
    from botocore.config import Config

    cfg = Config(retries={"max_attempts": 8, "mode": "adaptive"}, connect_timeout=5,
read_timeout=30)
    base = session.client("ec2", region_name="us-east-1", config=cfg)
    regions = sorted(r["RegionName"] for r in base.describe_regions()["Regions"])

    def scan(region: str):
        ec2 = session.client("ec2", region_name=region, config=cfg)
        inv = {"r": region, "run": 0, "stop": 0, "oth": 0, "types": {},
               "vols": 0, "gb": 0, "uvols": 0, "ugb": 0, "eip": 0, "ieip": 0}
        insts: List[dict] = []
        for page in ec2.get_paginator("describe_instances").paginate():
            for resv in page["Reservations"]:
```

```
            for i in resv["Instances"]:
                state = i["State"]["Name"]
                if state == "terminated":
                    continue
                name = next((t["Value"] for t in i.get("Tags", []) if t["Key"] == "Name"), "")
                if state == "running":
                    inv["run"] += 1
                    inv["types"][i["InstanceType"]] = inv["types"].get(i["InstanceType"], 0) + 1
                elif state == "stopped":
                    inv["stop"] += 1
                else:
                    inv["oth"] += 1
                insts.append({"id": i["InstanceId"], "n": name, "t": i["InstanceType"],
                              "st": state, "r": region})
        for page in ec2.get_paginator("describe_volumes").paginate():
            for v in page["Volumes"]:
                inv["vols"] += 1
                inv["gb"] += v.get("Size", 0)
                if not v.get("Attachments"):
                    inv["uvols"] += 1
                    inv["ugb"] += v.get("Size", 0)
        for a in ec2.describe_addresses()["Addresses"]:
            inv["eip"] += 1
            if not a.get("AssociationId"):
                inv["ieip"] += 1
        return inv, insts

    invs: List[dict] = []
    instances: List[dict] = []
    with ThreadPoolExecutor(max_workers=workers) as ex:
        futs = {ex.submit(scan, r): r for r in regions}
        for fut in as_completed(futs):
            try:
                inv, insts = fut.result()
            except Exception as exc:  # noqa: BLE001
                log.warning("Inventory skipped for %s: %s", futs[fut], exc)
                continue
            if inv["run"] + inv["stop"] + inv["oth"] + inv["vols"] + inv["eip"] > 0:
                invs.append(inv)
                instances.extend(insts)
    invs.sort(key=lambda i: (i["run"], i["stop"]), reverse=True)
    return invs, instances
```

```
# ----------------------------------------------------------------------------
# Analysis
# ----------------------------------------------------------------------------
def aggregate(costs: Dict[Tuple[str, str], Dict[date, float]],
              keyfn: Callable[[Tuple[str, str]], str]) -> Dict[str, Dict[date, float]]:
    """Collapse (service, region) -> any grouping (service, region, category)."""
    out: Dict[str, Dict[date, float]] = defaultdict(lambda: defaultdict(float))
    for key, days in costs.items():
        k = keyfn(key)
        for d, v in days.items():
            out[k][d] += v
    return out


def build_rows(costs: Dict[str, Dict[date, float]], today: date) -> List[Row]:
    yesterday = today - timedelta(days=1)
    first_this = today.replace(day=1)
    last_month_end = first_this - timedelta(days=1)
    last_month_start = last_month_end.replace(day=1)
    last7_start = today - timedelta(days=7)
    prev7_start, prev7_end = today - timedelta(days=14), today - timedelta(days=8)

    rows = []
    for name, days in costs.items():
        row = Row(
            service=name,
            last_month=period_sum(days, last_month_start, last_month_end),
            mtd=period_sum(days, first_this, yesterday),
            last7=period_sum(days, last7_start, yesterday),
            prev7=period_sum(days, prev7_start, prev7_end),
            yesterday=days.get(yesterday, 0.0),
        )
        if max(abs(row.last_month), abs(row.mtd), abs(row.last7), abs(row.yesterday)) >= 0.005:
            rows.append(row)
    rows.sort(key=lambda r: (r.mtd, r.last7, r.last_month), reverse=True)
    return rows


def sum_rows(name: str, rows: List[Row]) -> Row:
    return Row(
        service=name,
        last_month=sum(r.last_month for r in rows),
        mtd=sum(r.mtd for r in rows),
        last7=sum(r.last7 for r in rows),
```

```
        prev7=sum(r.prev7 for r in rows),
        yesterday=sum(r.yesterday for r in rows),
    )


def find_anomalies(costs: Dict[str, Dict[date, float]], today: date, pct: float, min_abs: float) ->
List[Anomaly]:
    yesterday = today - timedelta(days=1)
    base_start = yesterday - timedelta(days=7)
    out: List[Anomaly] = []
    for service, days in costs.items():
        y = days.get(yesterday, 0.0)
        baseline = period_sum(days, base_start, yesterday - timedelta(days=1)) / 7.0
        delta = y - baseline
        if delta >= min_abs and (baseline <= 0 or delta / baseline * 100 >= pct):
            out.append(Anomaly(service, y, baseline))
    out.sort(key=lambda a: a.delta, reverse=True)
    return out


def generate_report(ce, label: str, today: date, args, session=None) -> Report:
    first_last_month = (today.replace(day=1) - timedelta(days=1)).replace(day=1)
    start = min(first_last_month, today - timedelta(days=max(args.days, 15)))
    flt = build_filter(args.include_credits, args.tag)

    costs, unit = fetch_daily_costs(ce, start, today, flt)  # end exclusive -> through yesterday
    by_service = aggregate(costs, lambda k: k[0])
    by_region = aggregate(costs, lambda k: k[1])
    by_cat = aggregate(costs, lambda k: categorize(k[0]))

    rows = build_rows(by_service, today)
    region_rows = build_rows(by_region, today)
    cat_rows = build_rows(by_cat, today)
    total = sum_rows("TOTAL", rows)

    dates = list(day_range(start, today - timedelta(days=1)))
    daily = [0.0] * len(dates)
    series = []
    for (svc, region), days in costs.items():
        vals = [round(days.get(d, 0.0), 6) for d in dates]
        if any(v != 0 for v in vals):
            series.append({"s": svc, "r": region, "k": categorize(svc), "c": vals})
        for i, v in enumerate(vals):
            daily[i] += v
```

```
    report = Report(
        label=label, run_date=today, unit=unit, rows=rows, region_rows=region_rows, total=total,
        forecast_month_end=fetch_forecast(ce, today, flt),
        anomalies=find_anomalies(by_service, today, args.anomaly_pct, args.anomaly_min),
        dates=dates, daily=daily, series=series, cat_rows=cat_rows,
    )

    if args.ec2:
        try:
            report.ec2_types = fetch_ec2_types(ce, today, flt)
        except Exception as exc:  # noqa: BLE001
            log.warning("EC2 instance-type cost unavailable: %s", exc)

    res_map: Dict[str, dict] = {}
    if args.resources:
        try:
            res_map = fetch_resource_costs(ce, today, flt)
        except Exception as exc:  # noqa: BLE001
            log.warning("Resource-level cost unavailable (enable resource-level data in "
                        "Cost Explorer preferences): %s", exc)

    if args.inventory and session is not None:
        try:
            report.inventory, report.instances = fetch_inventory(session)
        except Exception as exc:  # noqa: BLE001
            log.warning("EC2 inventory unavailable: %s", exc)

    names = {i["id"]: i["n"] for i in report.instances}
    for i in report.instances:
        i["c"] = round(res_map.get(i["id"], {}).get("c", 0.0), 2) if res_map else None
    report.instances.sort(key=lambda i: (-(i["c"] or 0), i["st"] != "running", i["n"] or i["id"]))
    report.instances = report.instances[:300]

    kinds: Dict[str, dict] = {}
    for e in res_map.values():
        k = resource_kind(e["id"])
        e["k"], e["n"] = k, names.get(e["id"], "")
        agg = kinds.setdefault(k, {"k": k, "c": 0.0, "cnt": 0})
        agg["c"] += e["c"]
        agg["cnt"] += 1
    report.res_kinds = sorted(({"k": v["k"], "c": round(v["c"], 2), "cnt": v["cnt"]} for v in
kinds.values()),
                              key=lambda e: e["c"], reverse=True)
```

```
    top_res = sorted((e for e in res_map.values() if e["c"] >= 0.005), key=lambda e: e["c"],
reverse=True)[:40]
    report.resources = [{"id": e["id"], "k": e["k"], "r": e["r"], "c": round(e["c"], 2), "n": e["n"]}
                        for e in top_res]
    return report


def fleet_totals(report: Report) -> dict:
    inv = report.inventory
    return {
        "run": sum(i["run"] for i in inv), "stop": sum(i["stop"] for i in inv),
        "regions": sum(1 for i in inv if i["run"] + i["stop"] > 0),
        "uvols": sum(i["uvols"] for i in inv), "ugb": sum(i["ugb"] for i in inv),
        "ieip": sum(i["ieip"] for i in inv),
        "ec2_cost": sum(e["c"] for e in report.ec2_types),
    }


# ---------------------------------------------------------------------------
# Plain-text rendering (Slack / Teams)
# ---------------------------------------------------------------------------
def render_table(rows: List[Row], total: Row, top: int, first_col: str = "Service") -> str:
    w = 30
    head = f"{first_col:<{w}} {'LastMonth':>10} {'MTD':>10} {'Last7d':>10} {'Yday':>9} {'Now':>6}"
    lines = [head, "-" * len(head)]

    def fmt(r: Row, name: Optional[str] = None) -> str:
        wow = "n/a" if r.wow_pct is None else f"{r.wow_pct:+.0f}%"
        label = (name or r.service)[:w]
        return (f"{label:<{w}} {r.last_month:>10,.2f} {r.mtd:>10,.2f} "
                f"{r.last7:>10,.2f} {r.yesterday:>9,.2f} {wow:>6}")

    for r in rows[:top]:
        lines.append(fmt(r))
    rest = rows[top:]
    if rest:
        lines.append(fmt(sum_rows("", rest), f"Other ({len(rest)})"))
    lines.append("-" * len(head))
    lines.append(fmt(total, "TOTAL"))
    return "\n".join(lines)


def render_daily_bars(report: Report, n: int = 14) -> str:
    """Text bar chart of the last n days (works in Slack, Teams)."""
```

```
    dates, vals = report.dates[-n:], report.daily[-n:]
    if not vals:
        return ""
    mx = max(vals) or 1.0
    lines = []
    for d, v in zip(dates, vals):
        bar = "#" * max(1 if v > 0 else 0, round(v / mx * 24))
        lines.append(f"{d:%a %d %b}  {bar:<24} {money(v, report.unit)}")
    return "\n".join(lines)


def summary_lines(report: Report) -> List[str]:
    t, u = report.total, report.unit
    wow = "n/a" if t.wow_pct is None else f"{t.wow_pct:+.0f}% vs previous 7 days"
    lines = [
        f"Yesterday:        {money(t.yesterday, u)}",
        f"Last 7 days:      {money(t.last7, u)} ({wow})",
        f"Month to date:    {money(t.mtd, u)}",
        f"Last month:       {money(t.last_month, u)}",
    ]
    if report.forecast_month_end is not None:
        lines.append(f"Forecast (month): {money(t.mtd + report.forecast_month_end, u)}")
    return lines


def fleet_lines(report: Report) -> List[str]:
    if not report.inventory and not report.ec2_types:
        return []
    f = fleet_totals(report)
    lines = []
    if report.inventory:
        lines.append(f"EC2 running: {f['run']}   stopped: {f['stop']}   regions: {f['regions']}")
        lines.append(f"Unattached EBS: {f['uvols']} vol / {f['ugb']} GB   Idle Elastic IPs: {f['ieip']}")
    if report.ec2_types:
        lines.append(f"EC2 compute cost (last {report.ec2_days}d): {money(f['ec2_cost'],
report.unit)}")
        for e in report.ec2_types[:5]:
            lines.append(f"  {e['t']:<14} {e['r']:<16} {money(e['c'], report.unit)}")
    return lines


def _anomaly_lines(report: Report) -> List[str]:
    return [f"- {a.service}: {money(a.yesterday, report.unit)} (avg {money(a.baseline, report.unit)})"
            for a in report.anomalies[:5]]
```

```
def clip(text: str, limit: int) -> str:
    if len(text) <= limit:
        return text
    fence = "```" if text.rstrip().endswith("```") else ""   # keep the code block closed after
truncation
    return text[: limit - 20].rstrip() + "\n... (truncated)" + fence


def slack_payload(report: Report, top: int, dashboard_url: Optional[str] = None) -> dict:
    title = f"AWS Cost Report - {report.label} - {report.run_date:%a %d %b %Y}"

    def sec(text: str) -> dict:
        return {"type": "section", "text": {"type": "mrkdwn", "text": clip(text, 2990)}}

    blocks: List[dict] = [
        {"type": "header", "text": {"type": "plain_text", "text": title[:150]}},
        sec("```" + "\n".join(summary_lines(report)) + "```"),
        sec("*By service*\n```" + render_table(report.rows, report.total, top, "Service") + "```"),
        sec("*By category*\n```" + render_table(report.cat_rows, report.total, 10, "Category") +
"```"),
        sec("*By region*\n```" + render_table(report.region_rows, report.total, 6, "Region") + "```"),
    ]
    fl = fleet_lines(report)
    if fl:
        blocks.append(sec("*EC2 fleet*\n```" + "\n".join(fl) + "```"))
    blocks.append(sec("*Daily cost (last 14 days)*\n```" + render_daily_bars(report, 14) + "```"))
    if report.anomalies:
        blocks.append(sec("*:warning: Spikes yesterday (vs 7-day avg)*\n" +
"\n".join(_anomaly_lines(report))))
    if dashboard_url:
        blocks.append(sec(f"<{dashboard_url}|Open full dashboard>"))
    blocks.append({"type": "context", "elements": [
        {"type": "mrkdwn", "text": "aws_cost_report.py - Cost Explorer, unblended cost,
credits/refunds excluded"}]})
    return {"text": title, "blocks": blocks}


def teams_payload(report: Report, top: int, dashboard_url: Optional[str] = None) -> dict:
    title = f"AWS Cost Report - {report.label} - {report.run_date:%a %d %b %Y}"

    def block(text: str, **kw) -> dict:
        return {"type": "TextBlock", "text": text, "wrap": True, **kw}
```

```
    mono = dict(fontType="Monospace", size="Small", spacing="Medium", wrap=False)
    body = [
        block(title, weight="Bolder", size="Large"),
        block("\n".join(summary_lines(report)), fontType="Monospace", spacing="Small"),
        block("**By service**", spacing="Medium"),
        block(render_table(report.rows, report.total, top, "Service"), **mono),
        block("**By category**", spacing="Medium"),
        block(render_table(report.cat_rows, report.total, 10, "Category"), **mono),
        block("**By region**", spacing="Medium"),
        block(render_table(report.region_rows, report.total, 6, "Region"), **mono),
    ]
    fl = fleet_lines(report)
    if fl:
        body += [block("**EC2 fleet**", spacing="Medium"), block("\n".join(fl), **mono)]
    body += [block("**Daily cost (last 14 days)**", spacing="Medium"),
             block(render_daily_bars(report, 14), **mono)]
    if report.anomalies:
        body.append(block("\n".join(["**Spikes yesterday (vs 7-day avg)**"] +
_anomaly_lines(report)),
                          color="Attention", spacing="Medium"))
    card = {
        "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
        "type": "AdaptiveCard", "version": "1.4", "msteams": {"width": "Full"}, "body": body,
    }
    if dashboard_url:
        card["actions"] = [{"type": "Action.OpenUrl", "title": "Open dashboard", "url": dashboard_url}]
    return {"type": "message", "attachments": [{
        "contentType": "application/vnd.microsoft.card.adaptive", "contentUrl": None, "content":
card}]}


# ---------------------------------------------------------------------------
# Rich Linux terminal rendering (256-colour, unicode bars, heat calendar)
# ---------------------------------------------------------------------------
class T:
    color = False
    utf = True


PALETTE = [208, 75, 78, 141, 203, 80, 220, 205, 105, 149, 209, 246]
HEAT = [71, 178, 208, 167]   # green, yellow, orange, red
BOLD, DIM = "1", "2"
```

```
def fg(n: int) -> str:
    return f"38;5;{n}"


def bg(n: int) -> str:
    return f"48;5;{n}"


def paint(text: str, *codes: str) -> str:
    codes = tuple(c for c in codes if c)
    if not T.color or not codes:
        return text
    return "\033[" + ";".join(codes) + "m" + text + "\033[0m"


def g(utf_s: str, ascii_s: str) -> str:
    return utf_s if T.utf else ascii_s


def bar(frac: float, width: int, color: int = 208) -> str:
    frac = max(0.0, min(1.0, frac))
    if not T.utf:
        n = round(frac * width)
        return paint("#" * n, fg(color)) + paint("-" * (width - n), DIM)
    eighths = int(round(frac * width * 8))
    full, part = divmod(eighths, 8)
    s = "█" * full + ("▏▎▍▌▋▊▉"[part] if part and full < width else "")
    return paint(s, fg(color)) + paint("░" * (width - len(s)), fg(238))


def stackbar(parts: List[Tuple[str, float]], width: int) -> str:
    total = sum(v for _, v in parts if v > 0)
    if total <= 0:
        return ""
    out, acc, prev = "", 0.0, 0
    for i, (_, v) in enumerate(parts):
        acc += max(v, 0.0)
        end = round(acc / total * width)
        out += paint(g("█", "#") * (end - prev), fg(PALETTE[i % len(PALETTE)]))
        prev = end
    return out
```

```
def spark(vals: List[float]) -> str:
    blocks = g("▁▂▃▄▅▆▇█", "._-~=+*#")
    mx = max(vals) if vals else 0
    if mx <= 0:
        return ""
    q = quartiles(vals)
    out = ""
    for v in vals:
        ch = blocks[min(7, int(v / mx * 7.999))] if v > 0 else " "
        out += paint(ch, fg(HEAT[heat_level(v, q)])) if v > 0 else ch
    return out


def quartiles(vals: List[float]) -> Tuple[float, float, float]:
    pos = sorted(v for v in vals if v > 0)
    if not pos:
        return (0.0, 0.0, 0.0)
    q = lambda p: pos[min(len(pos) - 1, int(p * len(pos)))]  # noqa: E731
    return q(.25), q(.5), q(.75)


def heat_level(v: float, q: Tuple[float, float, float]) -> int:
    return 0 if v < q[0] else 1 if v < q[1] else 2 if v < q[2] else 3


def wow_cell(p: Optional[float], w: int = 7) -> str:
    if p is None:
        return paint(f"{'n/a':>{w}}", DIM)
    arrow = g("▲", "^") if p > 0.5 else g("▼", "v") if p < -0.5 else g("•", "-")
    col = 203 if p > 0.5 else 78 if p < -0.5 else 246
    return paint(f"{arrow}{abs(p):.0f}%".rjust(w), fg(col))


def section(title: str, width: int) -> str:
    rule = g("━", "=") * max(2, width - len(title) - 4)
    return paint(f"{g('━━', '==')} {title} ", BOLD, fg(214)) + paint(rule, fg(240))


def render_cards(items: List[tuple], width: int) -> str:
    n = len(items)
    w = max(18, (width - (n - 1)) // n)
    tl, tr, bl, br, h, v = g("╭", "+"), g("╮", "+"), g("╰", "+"), g("╯", "+"), g("─", "-"), g("│", "|")
    edge = paint(v, fg(240))
```

```
    def line(plain: str, painted: str) -> str:
        return edge + " " + painted + " " * max(0, w - 4 - len(plain)) + " " + edge

    rows: List[List[str]] = [[], [], [], [], []]
    for label, val, sub, col in items:
        lab, val, sub = label.upper()[:w - 4], val[:w - 4], sub[:w - 4]
        rows[0].append(paint(tl + h * (w - 2) + tr, fg(240)))
        rows[1].append(line(lab, paint(lab, DIM)))
        rows[2].append(line(val, paint(val, BOLD, fg(col))))
        rows[3].append(line(sub, paint(sub, DIM)))
        rows[4].append(paint(bl + h * (w - 2) + br, fg(240)))
    return "\n".join(" ".join(r) for r in rows)


def rich_table(rows: List[Row], total: Row, top: int, first_col: str, width: int, name_w: int = 24) ->
List[str]:
    fixed = 3 + name_w + 1 + 1 + 10 + 1 + 10 + 1 + 10 + 1 + 9 + 1 + 7 + 1 + 6
    bw = max(8, min(26, width - fixed))
    shown, rest = rows[:top], rows[top:]
    mx = max([r.mtd for r in shown] + [0.01])
    head = (f"{'#':>2} {first_col:<{name_w}} {'':<{bw}} {'Last month':>10} {'MTD':>10} "
            f"{'Last 7d':>10} {'Yday':>9} {'Now':>7} {'Share':>6}")
    rule = paint(g("—", "-") * len(head), fg(240))
    out = [paint(head, DIM), rule]

    def line(rank: str, name: str, r: Row, color: int, strong: bool = False, show_bar: bool = True) ->
str:
        nm = name[:name_w - 2].ljust(name_w - 2)
        share = r.mtd / total.mtd * 100 if total.mtd > 0 else 0.0
        return (f"{paint(rank.rjust(2), DIM)} {paint(g('●', '*'), fg(color))} {paint(nm, BOLD if strong else
'')} "
                f"{bar(r.mtd / mx, bw, color) if show_bar else ' ' * bw} "
                f"{r.last_month:>10,.2f} {paint(f'{r.mtd:>10,.2f}', BOLD)} {r.last7:>10,.2f}
{r.yesterday:>9,.2f} "
                f"{wow_cell(r.wow_pct)} {paint(f'{share:5.1f}%', DIM)}")

    for i, r in enumerate(shown):
        out.append(line(str(i + 1), r.service, r, PALETTE[i % len(PALETTE)]))
    if rest:
        out.append(line("", f"Other ({len(rest)})", sum_rows("", rest), 244))
    out += [rule, line("", "TOTAL", total, 255, strong=True, show_bar=False)]
    return out
```

```
def rich_regions(report: Report, top: int, width: int) -> List[str]:
    inv = {i["r"]: i for i in report.inventory}
    name_w = 16
    fixed = 3 + name_w + 1 + 1 + 11 + 10 + 8 + 6 + 6 + 9 + 6
    bw = max(8, min(22, width - fixed))
    rows = report.region_rows[:top]
    mx = max([r.mtd for r in rows] + [0.01])
    head = (f"{'#':>2} {'Region':<{name_w}} {'':<{bw}} {'MTD':>10} {'Yday':>9} {'Now':>7} "
            f"{'Run':>5} {'Stop':>5} {'EBS GB':>8} {'Idle*':>5}")
    out = [paint(head, DIM), paint(g("─", "-") * len(head), fg(240))]
    for i, r in enumerate(rows):
        c = PALETTE[i % len(PALETTE)]
        v = inv.get(r.service)
        if v:
            idle = v["uvols"] + v["ieip"]
            cols = (f"{v['run']:>5} {v['stop']:>5} {v['gb']:>8,} "
                    f"{paint(f'{idle:>5}', fg(220) if idle else DIM)}")
        else:
            cols = paint(f"{'-':>5} {'-':>5} {'-':>8} {'-':>5}", DIM)
        out.append(f"{paint(str(i + 1).rjust(2), DIM)} {paint(g('●', '*'), fg(c))} "
                   f"{r.service[:name_w - 2].ljust(name_w - 2)} {bar(r.mtd / mx, bw, c)} "
                   f"{paint(f'{r.mtd:>10,.2f}', BOLD)} {r.yesterday:>9,.2f} {wow_cell(r.wow_pct)} {cols}")
    if report.inventory:
        out.append(paint("   * idle = unattached EBS volumes + unassociated Elastic IPs
(~$3.65/mo each EIP)", DIM))
    return out


def rich_ec2(report: Report, width: int) -> List[str]:
    rows = report.ec2_types[:10]
    running = {(i["r"], t): n for i in report.inventory for t, n in i["types"].items()}
    mx = max([e["c"] for e in rows] + [0.01])
    bw = max(8, min(26, width - 74))
    head = f"{'':>2} {'Instance type':<16} {'Region':<16} {'':<{bw}} {f'Cost {report.ec2_days}d':>12}
{'Running':>8}"
    out = [paint(head, DIM), paint(g("─", "-") * len(head), fg(240))]
    for i, e in enumerate(rows):
        c = PALETTE[i % len(PALETTE)]
        n = running.get((e["r"], e["t"]))
        out.append(f"{paint(g('●', '*'), fg(c)):>2} {e['t'][:16]:<16} {e['r'][:16]:<16} {bar(e['c'] / mx, bw,
c)} "
                   f"{paint(f'{e['c']:>12,.2f}', BOLD)} {('-' if n is None else n):>8}")
    return out
```

```
def rich_resources(report: Report, width: int) -> List[str]:
    out: List[str] = []
    if report.res_kinds:
        tot = sum(k["c"] for k in report.res_kinds) or 1.0
        parts = [(k["k"], k["c"]) for k in report.res_kinds[:8]]
        out.append(stackbar(parts, min(width - 2, 100)))
        out.append("  ".join(f"{paint(g('●', '*'), fg(PALETTE[i % len(PALETTE)]))} {k} "
                             f"{paint(f'{v / tot * 100:.0f}%', DIM)}" for i, (k, v) in enumerate(parts)))
        out.append("")
    rows = report.resources[:12]
    mx = max([e["c"] for e in rows] + [0.01])
    bw = max(8, min(22, width - 95))
    head = (f"{'':>2} {'Kind':<17} {'Resource':<24} {'Name':<20} {'Region':<15} {'':<{bw}} "
            f"{f'Cost {report.res_days}d':>11}")
    out += [paint(head, DIM), paint(g("─", "-") * len(head), fg(240))]
    for i, e in enumerate(rows):
        c = PALETTE[i % len(PALETTE)]
        out.append(f"{paint(g('●', '*'), fg(c)):>2} {e['k'][:17]:<17} {e['id'].rsplit('/', 1)[-1][:24]:<24} "
                   f"{(e['n'] or '-')[:20]:<20} {e['r'][:15]:<15} {bar(e['c'] / mx, bw, c)} "
                   f"{paint(f'{e['c']:>11,.2f}', BOLD)}")
    return out


def rich_calendar(report: Report, weeks: int = 8) -> List[str]:
    """Week-by-week cost calendar: one coloured cell per day showing its cost,
    'Week N' row labels and a 'Color meaning' legend with the $ ranges."""
    pairs = list(zip(report.dates, report.daily))[-(weeks * 7):]
    if not pairs:
        return []
    q = quartiles([v for _, v in pairs])
    lookup = dict(pairs)
    dec = 2                                      # exact cents always ($1.56 stays $1.56, never $2)
    cw = len(f"${max(v for _, v in pairs):,.{dec}f}") + 3  # cell width
    money_s = lambda v: f"${v:,.{dec}f}"                   # noqa: E731
    d = pairs[0][0] - timedelta(days=pairs[0][0].weekday())
    names = ("Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun")
    out = [paint(" " * 15 + " ".join(f"{n.upper():^{cw}}" for n in names), DIM)]
    wk = 0
    while d <= pairs[-1][0]:
        wk += 1
        cells = []
        for i in range(7):
            v = lookup.get(d + timedelta(days=i))
```

```
            if v is None:
                cells.append(" " * cw)
            elif v <= 0:
                cells.append(paint(f"{money_s(0):^{cw}}", fg(250), bg(237)))
            else:
                cells.append(paint(f"{money_s(v):^{cw}}", fg(16), bg(HEAT[heat_level(v, q)])))
        out.append(paint(f"Week {wk:<2}", DIM) + paint(f" {d:%d %b} ", fg(240)) + " ".join(cells))
        d += timedelta(days=7)
    dash = g("\u2013", "-")
    labs = ("Low", "Medium", "High", "Very High")
    rng = (f"< {money_s(q[0])}", f"{money_s(q[0])}{dash}{money_s(q[1])}",
           f"{money_s(q[1])}{dash}{money_s(q[2])}", f"> {money_s(q[2])}")
    legend = "  ".join(paint("  ", bg(HEAT[i])) + f" {labs[i]} ({rng[i]})" for i in range(4))
    out.append("")
    out.append(paint("Color meaning:  ", BOLD) + legend)
    return out


def rich_all_services(report: Report, width: int) -> List[str]:
    """Every service with cost in the window + ACTIVE / IDLE status.
    ACTIVE = billed yesterday; IDLE = billed earlier in the window but not yesterday."""
    last_seen: Dict[str, date] = {}
    for s_ in report.series:
        for dt_, v in zip(report.dates, s_["c"]):
            if v >= 0.005 and (s_["s"] not in last_seen or dt_ > last_seen[s_["s"]]):
                last_seen[s_["s"]] = dt_
    nw = 36
    head = (f"{'#':>3} {'Service':<{nw}} {'Status':<9} {'MTD':>11} {'Last 7d':>11} "
            f"{'Yday':>10} {'Last billed':>12}")
    rule = paint(g("\u2500", "-") * len(head), fg(240))
    out = [paint(head, DIM), rule]
    active = 0
    for i, r in enumerate(report.rows):
        on = r.yesterday >= 0.005
        active += on
        status = paint(f"{g('\u25cf', '*')} ACTIVE", fg(78)) if on else paint(f"{g('\u25cb', 'o')} IDLE  ",
fg(246))
        lb = last_seen.get(r.service)
        out.append(f"{paint(str(i + 1).rjust(3), DIM)} {r.service[:nw]:<{nw}} {status} "
                   f"{paint(f'{r.mtd:>11,.2f}', BOLD)} {r.last7:>11,.2f} {r.yesterday:>10,.2f} "
                   f"{(f'{lb:%d %b}' if lb else '-'):>12}")
    n = len(report.rows)
    out += [rule, f"    {paint(str(active), BOLD, fg(78))} active   {paint(str(n - active), BOLD,
fg(246))} idle   "
```

```
            f"{paint(str(n), BOLD)} services   {paint('(active = billed yesterday)', DIM)}"]
    return out


def rich_all_instances(report: Report, width: int) -> List[str]:
    """Every EC2 instance with its state (running / stopped / other) and cost."""
    head = (f"{'#':>3} {'Name':<26} {'Instance ID':<20} {'Type':<13} {'Region':<14} "
            f"{'State':<10} {f'Cost {report.res_days}d':>11}")
    rule = paint(g("\u2500", "-") * len(head), fg(240))
    out = [paint(head, DIM), rule]
    for i, e in enumerate(report.instances):
        st = e["st"]
        col = 78 if st == "running" else 220 if st == "stopped" else 246
        cell = f"{g('\u25cf' if st == 'running' else '\u25cb', '*' if st == 'running' else 'o')} {st}"
        cost = "-" if e.get("c") is None else f"{e['c']:,.2f}"
        out.append(f"{paint(str(i + 1).rjust(3), DIM)} {(e['n'] or '-')[:26]:<26} {e['id']:<20}
{e['t'][:13]:<13} "
                   f"{e['r'][:14]:<14} {paint(f'{cell:<10}', fg(col))} {cost:>11}")
    run = sum(1 for e in report.instances if e["st"] == "running")
    stop = sum(1 for e in report.instances if e["st"] == "stopped")
    out += [rule, f"    {paint(str(run), BOLD, fg(78))} running   {paint(str(stop), BOLD, fg(220))}
stopped   "
                f"{paint(str(len(report.instances) - run - stop), BOLD)} other"]
    return out


def render_rich(report: Report, top: int, width: int) -> str:
    u, t = report.total, report.total
    unit = report.unit
    sep = g("·", "|")
    out: List[str] = []
    title = f"  AWS COST REPORT  {sep}  {report.label}  {sep}  {report.run_date:%a %d %b %Y}"
    out += [paint(title.ljust(width), BOLD, fg(16), bg(208)), ""]

    wow = t.wow_pct
    fc = None if report.forecast_month_end is None else t.mtd + report.forecast_month_end
    fc_sub = ""
    if fc is not None and t.last_month > 0:
        fc_sub = f"{(fc - t.last_month) / t.last_month * 100:+.0f}% vs last month"
    out.append(render_cards([
        ("Yesterday", money(t.yesterday, unit), f"{report.dates[-1]:%a %d %b}" if report.dates else
"", 214),
        ("Last 7 days", money(t.last7, unit), "n/a" if wow is None else f"{wow:+.0f}% vs prev 7d",
         203 if (wow or 0) > 0 else 78),
```

```
        ("Month to date", money(t.mtd, unit), f"{report.run_date:%B %Y}", 75),
        ("Last month", money(t.last_month, unit), "", 246),
        ("Forecast", "n/a" if fc is None else money(fc, unit), fc_sub, 141),
    ], width))
    out.append("")

    # --- daily trend + calendar
    out.append(section("DAILY COST", width))
    last30 = report.daily[-30:]
    if last30:
        pos = [v for v in last30 if v > 0]
        avg = sum(pos) / len(pos) if pos else 0.0
        out.append(f"  last {len(last30)}d  {spark(last30)}   "
                   f"{paint('min', DIM)} {money(min(last30), unit)}  {paint('avg', DIM)} {money(avg, unit)}
"
                   f"{paint('max', DIM)} {money(max(last30), unit)}")
        out.append("")
    out += ["  " + ln for ln in rich_calendar(report)]
    out.append("")

    # --- service / category / region
    out += [section("SERVICE-WISE", width)] + rich_table(report.rows, report.total, top, "Service",
width) + [""]
    out.append(section("CATEGORY-WISE", width))
    cat_total = sum(r.mtd for r in report.cat_rows)
    if cat_total > 0:
        parts = [(r.service, r.mtd) for r in report.cat_rows[:9]]
        out.append(stackbar(parts, min(width - 2, 100)))
        out.append("  ".join(f"{paint(g('●', '*'), fg(PALETTE[i % len(PALETTE)]))} {n} "
                             f"{paint(f'{v / cat_total * 100:.0f}%', DIM)}" for i, (n, v) in enumerate(parts)))
        out.append("")
    out += rich_table(report.cat_rows, report.total, 12, "Category", width) + [""]
    out += [section("REGION-WISE  +  EC2 FLEET", width)] + rich_regions(report, top, width) +
[""]

    # --- EC2
    if report.inventory:
        f = fleet_totals(report)
        out.append(section("EC2 FLEET", width))
        out.append(f"  {paint(str(f['run']), BOLD, fg(78))} running   {paint(str(f['stop']), BOLD,
fg(220))} stopped   "
                   f"{paint(str(f['regions']), BOLD)} regions   "
                   f"{paint(str(f['uvols']), BOLD, fg(220 if f['uvols'] else 246))} unattached volumes "
```

```
                f"({f['ugb']:,} GB)   {paint(str(f['ieip']), BOLD, fg(220 if f['ieip'] else 246))} idle Elastic
IPs")
        out.append("")
    if report.ec2_types:
        out += [section(f"EC2 COST BY INSTANCE TYPE (last {report.ec2_days}d)", width)] +
rich_ec2(report, width) + [""]
    if report.resources:
        out += [section(f"RESOURCE-LEVEL COST (EC2 + EBS/NAT/ENI, last
{report.res_days}d)", width)]
        out += rich_resources(report, width) + [""]

    out += [section("ALL SERVICES  (active / idle)", width)] + rich_all_services(report, width) +
[""]
    if report.instances:
        out += [section("ALL EC2 INSTANCES  (running / stopped)", width)] +
rich_all_instances(report, width) + [""]

    # --- alerts
    alerts: List[str] = []
    for a in report.anomalies[:5]:
        alerts.append(f"  {paint(g('▲', '^'), fg(203))} {a.service}: {paint(money(a.yesterday, unit),
BOLD)} "
                      f"(avg {money(a.baseline, unit)}, {paint('+' + money(a.delta, unit), fg(203))})")
    stopped = [i for i in report.instances if i["st"] == "stopped"]
    if stopped:
        alerts.append(f"  {paint(g('●', '*'), fg(220))} {len(stopped)} stopped instance(s) still pay for
their EBS volumes: "
                      + ", ".join((i["n"] or i["id"]) for i in stopped[:4]) + (" ..." if len(stopped) > 4 else ""))
    if report.inventory:
        f = fleet_totals(report)
        if f["uvols"]:
            alerts.append(f"  {paint(g('●', '*'), fg(220))} {f['uvols']} unattached EBS volume(s),
{f['ugb']:,} GB - delete or snapshot")
        if f["ieip"]:
            alerts.append(f"  {paint(g('●', '*'), fg(220))} {f['ieip']} idle Elastic IP(s) - each costs
~$3.65/month")
    if alerts:
        out += [section("ALERTS", width)] + alerts + [""]
    out.append(paint("  Cost Explorer, unblended cost; credits/refunds excluded unless
--include-credits", DIM))
    return "\n".join(out)


# ---------------------------------------------------------------------------
```

```
# HTML dashboard (self-contained: inline CSS/JS/SVG, no CDN, works offline)
# ---------------------------------------------------------------------------
HTML_TEMPLATE = r"""<!doctype html>
<html lang="en"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>AWS Cost Dashboard</title>
<style>
:root{--bg:#f4f5f8;--card:#fff;--ink:#1c2230;--mute:#6b7385;--line:#e4e7ee;--accent:#ff9900;--acc
ent2:#232f3e;
--low:#4fd8a4;--med:#fcd535;--high:#fb923c;--vhigh:#f87171;--empty:#eef0f4;--up:#dc2626;--do
wn:#16a34a}
@media(prefers-color-scheme:dark){:root{--bg:#0f1420;--card:#171e2e;--ink:#e8ecf5;--mute:#8
e98ae;--line:#263049;--empty:#1f2739;
--low:#2fa97c;--med:#c9a500;--high:#d9772b;--vhigh:#cf4f4f}}
*{box-sizing:border-box}body{margin:0;font:14px/1.45 system-ui,-apple-system,"Segoe
UI",Roboto,sans-serif;background:var(--bg);color:var(--ink)}
header{background:var(--accent2);color:#fff;padding:18px
28px;display:flex;flex-wrap:wrap;gap:12px;align-items:center;justify-content:space-between}
header h1{margin:0;font-size:20px}header h1 span{color:var(--accent)}header
small{color:#aab3c5}
main{max-width:1300px;margin:0 auto;padding:20px 24px 40px}
.kpis{display:grid;grid-template-columns:repeat(auto-fit,minmax(170px,1fr));gap:14px;margin-bot
tom:18px}
.kpi{background:var(--card);border:1px solid var(--line);border-radius:12px;padding:14px 16px}
.kpi .l{color:var(--mute);font-size:12px;text-transform:uppercase;letter-spacing:.04em}
.kpi .v{font-size:24px;font-weight:700;margin-top:2px}.kpi .s{font-size:12px;color:var(--mute)}
.up{color:var(--up)}.down{color:var(--down)}.mute{color:var(--mute)}
.panel{background:var(--card);border:1px solid var(--line);border-radius:12px;padding:16px
18px;margin-bottom:16px}
.bar{display:flex;flex-wrap:wrap;gap:10px;align-items:center;justify-content:space-between;marg
in-bottom:12px}
.tabs,.ranges{display:flex;gap:4px;flex-wrap:wrap}
button,select{font:inherit;color:var(--ink);background:var(--card);border:1px solid
var(--line);border-radius:8px;padding:6px 12px;cursor:pointer}
button.on{background:var(--accent);border-color:var(--accent);color:#1c1300;font-weight:600}
.tabs button.on{background:var(--accent2);border-color:var(--accent2);color:#fff}
.filters{display:flex;gap:8px;align-items:center;color:var(--mute);flex-wrap:wrap}
.grid2{display:grid;grid-template-columns:1fr
300px;gap:16px}@media(max-width:900px){.grid2{grid-template-columns:1fr}}
.cal{display:grid;grid-template-columns:44px repeat(7,1fr);gap:6px}
.cal .h{color:var(--mute);font-size:12px;text-align:center}.cal
.w{color:var(--mute);font-size:11px;align-self:center}
```

.cell{border-radius:8px;padding:5px
4px;text-align:center;min-height:52px;color:#10131a;display:flex;flex-direction:column;justify-con
tent:center}
.cell b{font-size:14px;white-space:nowrap}@media(max-width:640px){.cell
b{font-size:11px}.cal{gap:4px}}.cell
i{font-style:normal;font-size:10px;opacity:.7}.cell.e{background:var(--empty)}
.cell.m{outline:2px solid var(--accent2)}
.legend{display:flex;gap:14px;flex-wrap:wrap;margin-top:12px;color:var(--mute);font-size:12px}
.legend
span:before{content:"";display:inline-block;width:12px;height:12px;border-radius:3px;margin-righ
t:6px;vertical-align:-2px;background:var(--c)}
.ins{background:var(--bg);border:1px solid var(--line);border-radius:10px;padding:12px
14px;margin-bottom:10px}
.ins .l{color:var(--mute);font-size:12px}.ins .v{font-weight:700;font-size:16px}
svg text{fill:var(--mute);font-size:11px}svg .grid{stroke:var(--line)}
.row{display:grid;grid-template-columns:200px 1fr
190px;gap:10px;align-items:center;margin:7px 0}
.row .n{white-space:nowrap;overflow:hidden;text-overflow:ellipsis}.row
.t{background:var(--empty);border-radius:6px;height:14px;overflow:hidden;display:flex}
.row
.f{height:100%;border-radius:6px;background:linear-gradient(90deg,var(--accent),#ffb84d)}.row
.a{text-align:right;font-variant-numeric:tabular-nums}
@media(max-width:700px){.row{grid-template-columns:110px 1fr 110px}}
table{border-collapse:collapse;width:100%;font-variant-numeric:tabular-nums}th,td{padding:7px
10px;text-align:right;border-bottom:1px solid var(--line)}
th:first-child,td:first-child{text-align:left}th{color:var(--mute);font-size:12px;font-weight:600}tfoot
td{font-weight:700}
.wrap{overflow-x:auto}.dn{display:flex;gap:24px;align-items:center;flex-wrap:wrap}.dn .lg
div{margin:3px 0}
.dot{display:inline-block;width:10px;height:10px;border-radius:50%;margin-right:6px}
h3{margin:0 0 10px;font-size:15px}.warn{border-left:4px solid var(--up)}
.cards2{display:grid;grid-template-columns:repeat(auto-fit,minmax(440px,1fr));gap:16px}.cards2
.wide{grid-column:1/-1}
@media(max-width:520px){.cards2{grid-template-columns:1fr}}
.mini{display:grid;grid-template-columns:repeat(auto-fit,minmax(140px,1fr));gap:10px;margin-bot
tom:16px}
.mini div{background:var(--bg);border:1px solid var(--line);border-radius:10px;padding:10px
12px}
.mini .l{color:var(--mute);font-size:11px;text-transform:uppercase}.mini
.v{font-size:20px;font-weight:700}
.badge{display:inline-block;padding:1px
8px;border-radius:10px;font-size:11px;background:var(--empty)}
.badge.run{background:#16a34a22;color:var(--down)}.badge.stop{background:#dc262622;color:
var(--up)}

```
.hm td,.hm th{padding:6px 8px;font-size:12px}.hm td{text-align:center;border:1px solid
var(--bg)}.hm th:first-child,.hm td:first-child{text-align:left}
.two{display:grid;grid-template-columns:1fr
1fr;gap:20px}@media(max-width:900px){.two{grid-template-columns:1fr}}
</style></head><body>
<header><h1><span>AWS</span> Cost Dashboard &middot; <span
id="lbl"></span></h1><small id="gen"></small></header>
<main>
<div class="kpis" id="kpis"></div>
<div id="anoms"></div>
<div class="panel">
 <div class="bar">
  <div class="tabs" id="tabs"></div>
  <div class="filters">
   <div class="ranges" id="ranges"></div>
   <select id="fsvc"></select><select id="freg"></select>
  </div>
 </div>
 <div id="view"></div>
</div>
<div class="cards2" id="ins"></div>
<div class="panel"><div class="bar"><h3>Period summary (unfiltered)</h3></div><div
class="wrap" id="period"></div></div>
</main>
<script>
const D=__DATA__;
const S={tab:'calendar',range:30,svc:'',reg:''};
const
COL=['#ff9900','#3b82f6','#10b981','#a855f7','#ef4444','#14b8a6','#eab308','#ec4899','#6366f1','
#84cc16','#f97316','#64748b'];
const $=id=>document.getElementById(id);
const
esc=s=>String(s).replace(/[&<>"']/g,c=>({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;',"'":'&#39;'}[c]));
/* nf = exact money: always cents, never rounded to whole dollars (so $1.56 stays $1.56) */
const nf=(n,d=2)=>{const
r=Number(n.toFixed(d));return(r<0?'-':'')+(D.unit==='USD'?'$':'')+Math.abs(r).toLocaleString(und
efined,{minimumFractionDigits:d,maximumFractionDigits:d})};
const short=n=>(D.unit==='USD'?'$':'')+(n>=1000?(n/1000).toFixed(1)+'k':n.toFixed(n<10?1:0));
const MON=['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec'];
const DAY=['Mon','Tue','Wed','Thu','Fri','Sat','Sun'];
const dow=s=>(new Date(s+'T00:00:00Z').getUTCDay()+6)%7;
const lab=s=>{const p=s.split('-');return +p[2]+' '+MON[+p[1]-1]};
const pct=v=>v==null?'n/a':(v>=0?'+':'')+v.toFixed(0)+'%';
const sum=(a,k)=>a.reduce((x,y)=>x+(y[k]||0),0);
```

```
function view(){
  const N=D.dates.length,n=S.range==='all'?N:Math.min(S.range,N),lo=N-n;
  const dates=D.dates.slice(lo),daily=new Array(n).fill(0),bs={},br={},bk={};
  for(const s of D.series){
    if(S.svc&&s.s!==S.svc)continue;if(S.reg&&s.r!==S.reg)continue;
    let t=0;for(let i=0;i<n;i++){const v=s.c[lo+i];daily[i]+=v;t+=v}
    bs[s.s]=(bs[s.s]||0)+t;br[s.r]=(br[s.r]||0)+t;bk[s.k]=(bk[s.k]||0)+t;
  }
  const rank=o=>Object.entries(o).filter(e=>Math.abs(e[1])>=0.005).sort((a,b)=>b[1]-a[1]);
  return{dates,daily,svc:rank(bs),reg:rank(br),cat:rank(bk),total:daily.reduce((a,b)=>a+b,0)};
}

function kpis(){
  const t=D.total,w=t.wow;
  const items=[['Yesterday',nf(t.yesterday),D.dates[D.dates.length-1]],
   ['Last 7 days',nf(t.last7),`<span class="${w>0?'up':'down'}">{pct(w)}</span> vs previous 7d`],
   ['Month to date',nf(t.mtd),D.month],['Last month',nf(t.last_month),''],
   ['Forecast (month end)',D.forecast==null?'n/a':nf(D.forecast),'Cost Explorer forecast']];
  if(D.inv.length){const r=sum(D.inv,'run'),s=sum(D.inv,'stop');
   items.push(['EC2 running',r,`${s} stopped &middot; ${D.inv.filter(i=>i.run+i.stop>0).length} regions`]);}
  ('kpis').innerHTML = items.map(i => ` <divclass = "kpi" ><divclass = "l" >{i[0]}</div><div class="v">i[1] </div><divclass = "s" >{i[2]}</div></div>`).join('');
  $('anoms').innerHTML=D.anomalies.length?`<div class="panel warn"><h3>Spikes yesterday (vs 7-day average)</h3>`+
   D.anomalies.map(a=>`<div>esc(a.s): b >{nf(a.y)}</b> (avg nf(a.b), <spanclass = "up" >+{nf(a.y-a.b)}</span>)</div>`).join('')+`</div>`:'';
}

function calendar(v){
  const vals=v.daily.filter(x=>x>0).sort((a,b)=>a-b);
  const q=p=>vals.length?vals[Math.min(vals.length-1,Math.floor(p*vals.length))]:0;
  const t1=q(.25),t2=q(.5),t3=q(.75);
  const cls=x=>x<=0?'e':x<t1?'var(--low)':x<t2?'var(--med)':x<t3?'var(--high)':'var(--vhigh)';
  let cells='',first=dow(v.dates[0]);
  cells+=['','Mon','Tue','Wed','Thu','Fri','Sat','Sun'].map(h=>`<div class="h">${h}</div>`).join('');
  let col=first,wk=1,html=`<div class="w">W${wk}</div>`+'<div></div>'.repeat(first);
  v.dates.forEach((d,i)=>{
    if(col===7){col=0;wk++;html+=`<div class="w">W${wk}</div>`}
    const c=v.daily[i],bg=cls(c),p=d.split('-');
    /* exact cost in every cell (cents kept), same number as the tooltip */
```

```
    html+=`<div class="cell ${c<=0?'e':''} p[2] === '01'?'m':''"style = "
{c>0?'background:'+bg:''}" title="${d}: nf(c)" >< i >
{lab(d)}</i><b>${Math.abs(c)>=0.005?nf(c):'-'}</b></div>`;
    col++;
  });
  const wd=[],we=[];v.dates.forEach((d,i)=>{if(v.daily[i]>0)(dow(d)>=5?we:wd).push(v.daily[i])});
  const avg=a=>a.length?a.reduce((x,y)=>x+y,0)/a.length:0;
  const hi=v.daily.indexOf(Math.max(...v.daily)),wl=avg(wd)?(1-avg(we)/avg(wd))*100:0;
  const top=v.svc[0];
  const ins=[['Highest spend on',`${DAY[dow(v.dates[hi])]}, ${lab(v.dates[hi])}`,nf(v.daily[hi])],
   ['Weekend spend is',`${Math.abs(wl).toFixed(0)}% ${wl>=0?'lower':'higher'}`,'than weekdays'],
   ['Top service',top?esc(top[0]):'-',top&&v.total?(top[1]/v.total*100).toFixed(0)+'% of selected
spend':''],
   ['Top region',v.reg[0]?esc(v.reg[0][0]):'-',v.reg[0]&&v.total?(v.reg[0][1]/v.total*100).toFixed(0)+'%
of selected spend':''],
   ['Total of selected days',nf(v.total),'sum of the '+v.dates.length+' calendar cells'],
   ['Average per day',nf(v.total/Math.max(1,v.daily.filter(x=>x>0).length)),'over '+v.dates.length+'
days']];
  return `<div class="grid2"><div><div class="cal">cells{html}</div>
  <div class="legend"><span style="--c:var(--low)">Low (&lt;
nf(t1)) < /span >< spanstyle = " -- c: var(-- med)" > Medium({nf(t1)}-${nf(t2)})</span>
  <span style="--c:var(--high)">High (nf(t2) −{nf(t3)})</span><span
style="--c:var(--vhigh)">Very high (&gt; ${nf(t3)})</span></div></div>
  <div>${ins.map(i=>`<div class="ins"><div class="l">${i[0]}</div><div class="v">
i[1] < /div >< divclass = "l" >{i[2]}</div></div>`).join('')}</div></div>`;
}

function trend(v){
  const W=1000,H=320,p={l:56,r:14,t:14,b:30},n=v.daily.length;
  const mx=Math.max(...v.daily,1)*1.1;
  const x=i=>p.l+(n>1?i*(W-p.l-p.r)/(n-1):0),y=c=>p.t+(H-p.t-p.b)*(1-c/mx);
  let g='';for(let k=0;k<=4;k++){const val=mx*k/4;g+=`<line class="grid" x1="p. l"x2 = "{W-p.r}"
y1="y(val)"y2 = "{y(val)}"/><text x="p. l − 8"y = "{y(val)+4}"
text-anchor="end">${short(val)}</text>`}
  const step=Math.max(1,Math.ceil(n/9));
  for(let i=0;i<n;i+=step)g+=`<text x="x(i)"y = "{H-8}"
text-anchor="middle">${lab(v.dates[i])}</text>`;
  const pts=v.daily.map((c,i)=>x(i)+','+y(c)).join(' ');
  const ma=v.daily.map((c,i)=>{const a=v.daily.slice(Math.max(0,i-6),i+1);return
x(i)+','+y(a.reduce((s,z)=>s+z,0)/a.length)}).join(' ');
  const dots=v.daily.map((c,i)=>`<circle cx="x(i)"cy = "{y(c)}" r="
n > 120? 2: 3. 5"fill = "#ff9900" >< title >{v.dates[i]}: ${nf(c)}</title></circle>`).join('');
  return `<svg viewBox="0 0 ${W} H"width = "100%" >{g}<polygon points="p. l,{y(0)} ${pts}
x(n − 1),{y(0)}" fill="#ff9900" opacity=".12"/>
```

```
  <polyline points="
pts"fill = "none"stroke = "#ff9900"stroke − width = "2"/ >< polylinepoints = "{ma}"
fill="none" stroke="#3b82f6" stroke-width="2.5" stroke-dasharray="6 4"/>${dots}</svg>
  <div class="legend"><span style="--c:#ff9900">Daily cost</span><span
style="--c:#3b82f6">7-day average</span></div>`;
}

function bars(list,total){
  if(!list.length)return'<p class="mute">No data.</p>';
  const mx=list[0][1]||1;
  return list.slice(0,15).map((e,i)=>`<div class="row"><div class="n" title="
esc(e[0])" >< spanclass = "dot"style = "background:
{COL[i%COL.length]}"></span>${esc(e[0])}</div>
  <div class="t"><div class="f" style="width:Math. max(1, e[1]/mx * 100)%; background:
{COL[i%COL.length]}"></div></div>
  <div class="a">${nf(e[1])} &middot; ${total?(e[1]/total*100).toFixed(1):0}%</div></div>`).join('');
}
function hb(list,f){ // generic horizontal bars: list=[[label,value],...]
  if(!list.length)return'<p class="mute">No data.</p>';
  const mx=Math.max(...list.map(e=>e[1]),0.0001);
  return list.map((e,i)=>`<div class="row"><div class="n" title="
esc(e[0])" >< spanclass = "dot"style = "background:
{COL[i%COL.length]}"></span>${esc(e[0])}</div>
  <div class="t"><div class="f" style="width:Math. max(1, e[1]/mx * 100)%; background:
{COL[i%COL.length]}"></div></div><div class="a">${f(e)}</div></div>`).join('');
}
function donut(list,total,f=nf,c=nf){
  if(!list.length||!total)return'<p class="mute">No data.</p>';
  const
top=list.slice(0,8),rest=list.slice(8).reduce((a,e)=>a+e[1],0);if(rest>0)top.push(['Other',rest]);
  const r=70,C=2*Math.PI*r;let off=0,segs='';
  top.forEach((e,i)=>{const len=total?e[1]/total*C:0;segs+=`<circle r="
r"cx = "100"cy = "100"fill = "none"stroke = "{COL[i%COL.length]}" stroke-width="30"
stroke-dasharray="${len} C − len"stroke − dashoffset = "{-off}" transform="rotate(-90 100
100)"><title>${esc(e[0])}: ${f(e[1])}</title></circle>`;off+=len});
  return `<div class="dn"><svg viewBox="0 0 200 200" width="190"><circle r="
r"cx = "100"cy = "100"fill = "none"stroke = "var(−− empty)"stroke − width = "30"/ >
{segs}
  <text x="100" y="98" text-anchor="middle"
style="font-size:15px;font-weight:700;fill:var(--ink)">${c(total)}</text><text x="100" y="116"
text-anchor="middle">total</text></svg>
  <div class="lg">${top.map((e,i)=>`<div><span class="dot"
style="background:${COL[i%COL.length]}"></span>esc(e[0]) < b >
{total?(e[1]/total*100).toFixed(0):0}%</b></div>`).join('')}</div></div>`;
```

```
}

function dailyStack(v){ // stacked daily bars by top services
  const W=1000,H=300,p={l:56,r:14,t:10,b:30},n=v.dates.length,N=D.dates.length,lo=N-n;
  const top=v.svc.slice(0,6).map(e=>e[0]);
  const m=top.map(()=>new Array(n).fill(0)),oth=new Array(n).fill(0);
  for(const s of D.series){if(S.svc&&s.s!==S.svc)continue;if(S.reg&&s.r!==S.reg)continue;
    const k=top.indexOf(s.s);for(let i=0;i<n;i++){if(k>=0)m[k][i]+=s.c[lo+i];else oth[i]+=s.c[lo+i]}}
  const mx=Math.max(...v.daily,1)*1.1,bw=(W-p.l-p.r)/n,y=c=>p.t+(H-p.t-p.b)*(1-c/mx);
  let g='';for(let k=0;k<=4;k++){const val=mx*k/4;g+=`<line class="grid" x1="$p.l"x2 = "{W-p.r}"
y1="$y(val)"y2 = "{y(val)}"/><text x="$p.l - 8"y = "{y(val)+4}"
text-anchor="end">${short(val)}</text>`}
  const step=Math.max(1,Math.ceil(n/9));for(let i=0;i<n;i+=step)g+=`<text x="
$p.l + i * bw + bw/2"y = "{H-8}" text-anchor="middle">${lab(v.dates[i])}</text>`;
  let rects='';for(let i=0;i<n;i++){let
acc=0;[...m.map((a,k)=>[a[i],COL[k]]),[oth[i],'#94a3b8']].forEach(([val,c])=>{if(val>0){rects+=`<rec
t x="$p.l + i * bw + bw *.1"y = "{y(acc+val)}" width="$bw *.8"height = "{y(acc)-y(acc+val)}"
fill="${c}"/>`;acc+=val}})}
  return `<svg viewBox="0 0 ${W} $H"width = "100%" >{g}
$rects < /svg >< divclass = "legend" >{top.map((t,k)=>`<span style="--c:$COL[k]" >
{esc(t)}</span>`).join('')}<span style="--c:#94a3b8">Other</span></div>`;
}

/* ---------- EC2 / resources ---------- */
function ec2tab(){
  const inv=D.inv;
  if(!inv.length&&!D.ec2t.length)return'<p class="mute">No EC2 data. Inventory needs
ec2:Describe* permissions (or it was disabled with --no-inventory / --no-ec2).</p>';
  const run=sum(inv,'run'),stop=sum(inv,'stop'),ugb=sum(inv,'ugb'),ieip=sum(inv,'ieip');
  const ec2cost=D.ec2t.reduce((a,e)=>a+e.c,0);
  const types={};inv.forEach(i=>{for(const t in i.types)types[t]=(types[t]||0)+i.types[t]});
  const tl=Object.entries(types).sort((a,b)=>b[1]-a[1]),tt=tl.reduce((a,e)=>a+e[1],0);
  const mini=[['Running',run],['Stopped',stop],['Instance types',tl.length],[`EC2 compute
(${D.ec2days}d)`,nf(ec2cost)],['Unattached EBS',ugb.toLocaleString()+' GB'],['Idle Elastic
IPs',ieip]];
  const regs=inv.filter(i=>i.run+i.stop+i.oth>0).sort((a,b)=>b.run-a.run);
  const mxr=Math.max(...regs.map(i=>i.run+i.stop+i.oth),1);
  const fleet=regs.map(i=>`<div class="row"><div class="n">
$esc(i.r) < /div >< divclass = "t" >< divclass = "f"style = "width:
{i.run/mxr*100}%;background:#16a34a;border-radius:0"></div><div class="f" style="width:
$(i.stop + i.oth)/mxr * 100%; background: #94a3b8; border - radius: 0" >< /div >< /div >< divclass =
{i.run} running &middot; ${i.stop} stopped</div></div>`).join('');
  const ins=D.inst.slice(0,40);
```

```
  return `<div class="mini">${mini.map(m=>`<div><div class="l">${m[0]}</div><div
class="v">${m[1]}</div></div>`).join('')}</div>
  <div class="two"><div><h3>Instances per region</h3>${fleet||'<p class="mute">No
instances.</p>'}
   <div class="legend"><span style="--c:#16a34a">Running</span><span
style="--c:#94a3b8">Stopped / other</span></div></div>
  <div><h3>Running instances by type</h3>${donut(tl,tt,e=>e+'
instances',e=>Math.round(e))}</div></div>
  <h3 style="margin-top:18px">EC2 compute cost by instance type &middot; region
(${D.ec2days}d)</h3>
  ${hb(D.ec2t.slice(0,15).map(e=>[e.t+' · '+e.r,e.c]),e=>nf(e[1]))}
  <h3 style="margin-top:18px">Instances${D.inst.length&&D.inst[0].c!=null?` (by cost, last
${D.resdays}d)`:''}</h3>
  <div
class="wrap"><table><thead><tr><th>Name</th><th>Instance</th><th>Type</th><th>State</t
h><th>Region</th><th>Cost</th></tr></thead><tbody>${
   ins.map(i=>`<tr><td>esc(i.n||' − ') < /td >< td >
{esc(i.id)}</td><td>${esc(i.t)}</td><td><span class="badge
i.st === 'running'?'run': i.st === 'stopped'?'stop':'' >{esc(i.st)}</span></td><td>
esc(i.r) < /td >< td >{i.c==null?'-':nf(i.c)}</td></tr>`).join('')||'<tr><td
colspan="6">None</td></tr>'}</tbody></table></div>`;
}

function restab(){
  if(!D.res.length&&!D.inv.length)return'<p class="mute">No resource-level data. Enable
resource-level data in Cost Explorer preferences and run without --no-resources /
--no-inventory.</p>';
  const tot=D.reskinds.reduce((a,e)=>a+e.c,0),rm={};D.regions.forEach(r=>rm[r.n]=r);
  const kinds=D.reskinds.map(k=>[k.k,k.c]);
  const rows=D.inv.map(i=>{const r=rm[i.r];return `<tr><td>esc(i.r) < /td >< td >
{r?nf(r.mtd):'-'}</td><td>i.run < /td >< td >{i.stop}</td><td>i.vols < /td >< td >
{i.gb.toLocaleString()}</td><td>i.ugb.toLocaleString() < /td >< td >
{i.ieip}</td><td>${r&&i.run?nf(r.mtd/i.run):'-'}</td></tr>`}).join('');
  return `<div class="two"><div><h3>Cost by resource kind (D.resdaysd) < /h3 >
{donut(kinds,tot)}</div>
   <div><h3>Most expensive resources (D.resdaysd) < /h3 >
{hb(D.res.slice(0,12).map(e=>[(e.n?e.n+' · ':'')+e.id.split('/').pop()+'
('+e.k+')',e.c]),e=>nf(e[1]))}</div></div>
  ${D.inv.length?`<h3 style="margin-top:18px">Resources per region</h3><div
class="wrap"><table><thead><tr><th>Region</th><th>MTD
cost</th><th>Running</th><th>Stopped</th><th>EBS vols</th><th>EBS
GB</th><th>Unattached GB</th><th>Idle EIPs</th><th>MTD / running
inst.</th></tr></thead><tbody>${rows}</tbody></table></div>`:''}`;
}
```

```
/* ---------- insight cards below ---------- */
function cumul(){
  const N=D.dates.length,tot=new Array(N).fill(0);
  for(const s of D.series){if(S.svc&&s.s!==S.svc)continue;if(S.reg&&s.r!==S.reg)continue;for(let
i=0;i<N;i++)tot[i]+=s.c[i]}
  const mk=m=>{const a=new Array(31).fill(null);let
acc=0;D.dates.forEach((d,i)=>{if(d.slice(0,7)===m){acc+=tot[i];a[+d.slice(8,10)-1]=acc}});return
a};
  const
cur=mk(D.cur_month),hasPrev=D.dates[0]<=D.prev_month+'-01',prv=hasPrev?mk(D.prev_mont
h):[];
  const W=520,H=250,p={l:52,r:12,t:12,b:26};
  const all=[...cur,...prv].filter(z=>z!=null),mx=Math.max(...all,1)*1.1;
  const X=i=>p.l+i*(W-p.l-p.r)/30,Y=c=>p.t+(H-p.t-p.b)*(1-c/mx);
  const line=(a,col,dash)=>{const pts=a.map((c,i)=>c==null?null:X(i)+','+Y(c)).filter(Boolean);
    return pts.length?`<polyline points="pts.join('')"fill = "none"stroke = "{col}"
stroke-width="2.5" ${dash?'stroke-dasharray="6 4"':''}/>`:''};
  let g='';for(let k=0;k<=4;k++){const val=mx*k/4;g+=`<line class="grid" x1="p.l"x2 = "{W-p.r}"
y1="Y(val)"y2 = "{Y(val)}"/><text x="p.l − 6"y = "{Y(val)+4}"
text-anchor="end">${short(val)}</text>`}
  for(const d of [1,8,15,22,29])g+=`<text x="X(d − 1)"y = "{H-6}"
text-anchor="middle">${d}</text>`;
  return `<svg viewBox="0 0 ${W} H"width = "100%" >{g}line(prv,'#3b82f6',true)
{line(cur,'#ff9900')}</svg>
  <div class="legend"><span style="--c:#ff9900">This month</span><span
style="--c:#3b82f6">Last month</span><span>day of month</span></div>`;
}
function weekday(v){
  const s=Array(7).fill(0),c=Array(7).fill(0);
  v.dates.forEach((d,i)=>{if(v.daily[i]>0){const k=dow(d);s[k]+=v.daily[i];c[k]++}});
  return hb(s.map((x,i)=>[DAY[i],c[i]?x/c[i]:0]),e=>nf(e[1]));
}
function heat(v){
  const rs=v.reg.slice(0,8).map(e=>e[0]),ss=v.svc.slice(0,8).map(e=>e[0]);
  if(!rs.length||!ss.length)return'<p class="mute">No data.</p>';
  const N=D.dates.length,lo=N-v.dates.length,m={};
  for(const s of D.series){if(S.svc&&s.s!==S.svc)continue;if(S.reg&&s.r!==S.reg)continue;
    if(!rs.includes(s.r)||!ss.includes(s.s))continue;let t=0;for(let
i=lo;i<N;i++)t+=s.c[i];m[s.s+'|'+s.r]=(m[s.s+'|'+s.r]||0)+t}
  const mx=Math.max(1,...Object.values(m));
  const cell=val=>{if(!(val>=0.005))return'<td class="mute">-</td>';const
a=0.1+0.85*Math.sqrt(val/mx);
```

```
   return `<td style="background:rgba(255,153,0,a.toFixed(2));{a>0.55?'color:#10131a':''}"
title="nf(val)" >{nf(val)}</td>`};
  return `<div class="wrap"><table class="hm"><thead><tr><th>Service \\
Region</th>${rs.map(r=>`<th>${esc(r)}</th>`).join('')}</tr></thead><tbody>${
   ss.map(s=>`<tr><td>esc(s) < /td >
{rs.map(r=>cell(m[s+'|'+r])).join('')}</tr>`).join('')}</tbody></table></div>`;
}
function movers(){
  const
mv=D.services.map(r=>[r.n,r.l7-r.p7]).filter(e=>Math.abs(e[1])>=0.005).sort((a,b)=>Math.abs(b[1]
)-Math.abs(a[1])).slice(0,10);
  if(!mv.length)return'<p class="mute">No data.</p>';
  const mx=Math.max(...mv.map(e=>Math.abs(e[1])),0.0001);
  return mv.map(e=>`<div class="row"><div class="n" title="esc(e[0])" >{esc(e[0])}</div>
   <div class="t"><div class="f" style="width:
Math.max(1, Math.abs(e[1])/mx * 100)%; background:
{e[1]>0?'var(--up)':'var(--down)'}"></div></div>
   <div class="a e[1] > 0? 'up': 'down'" >{e[1]>0?'+':'-'}${nf(Math.abs(e[1]))}</div></div>`).join('');
}
function insights(v){
  const cards=[['Month-to-date vs last month (cumulative)',cumul()],['Average cost by
weekday',weekday(v)],
   ['Spend by category',donut(v.cat,v.total)],['Biggest movers: last 7d vs previous 7d',movers()],
   ['Region × service heatmap',heat(v),1]];
  if(D.inv.length)cards.push(['Running EC2 instances by
region',hb(D.inv.filter(i=>i.run>0).sort((a,b)=>b.run-a.run).map(i=>[i.r,i.run]),e=>e[1])]);
  $('ins').innerHTML=cards.map(c=>`<div class="panel c[2]? 'wide': '" >< h3 >
{c[0]}</h3>${c[1]}</div>`).join('');
}

function render(){
  const v=view();let h='';
  if(S.tab==='calendar')h=calendar(v);
  else if(S.tab==='trend')h=`<h3>Daily cost trend</h3>
trend(v) < h3style = "margin − top: 18px" > Dailycostbytopservices < /h3 >
{dailyStack(v)}`;
  else if(S.tab==='services')h=`<div class="grid2" style="grid-template-columns:260px 1fr"><div>
donut(v.svc, v.total) < /div >< div >{bars(v.svc,v.total)}</div></div>`;
  else if(S.tab==='categories')h=`<div class="grid2" style="grid-template-columns:260px
1fr"><div>donut(v.cat, v.total) < /div >< div >{bars(v.cat,v.total)}</div></div>`;
  else if(S.tab==='regions')h=`<div class="grid2" style="grid-template-columns:260px 1fr"><div>
donut(v.reg, v.total) < /div >< div >{bars(v.reg,v.total)}</div></div>`;
  else if(S.tab==='ec2')h=ec2tab();
  else if(S.tab==='resources')h=restab();
```

```
  else{const t=(l,list)=>`<h3>
l < /h3 >< divclass = "wrap" >< table >< thead >< tr >< th >{l.replace('By
','')}</th><th>Cost</th><th>Share</th></tr></thead><tbody>${list.map(e=>`<tr><td>${esc(e[0])}
</td><td>nf(e[1]) < /td >< td >
{v.total?(e[1]/v.total*100).toFixed(1):0}%</td></tr>`).join('')}</tbody><tfoot><tr><td>Total</td><td
>${nf(v.total)}</td><td></td></tr></tfoot></table></div>`;
    h=t('By service',v.svc)+'<br>'+t('By category',v.cat)+'<br>'+t('By region',v.reg)}
  $('view').innerHTML=h;
  insights(v);
  document.querySelectorAll('#tabs
button').forEach(b=>b.classList.toggle('on',b.dataset.k===S.tab));
  document.querySelectorAll('#ranges
button').forEach(b=>b.classList.toggle('on',b.dataset.k==String(S.range)));
}

function period(){
  const tb=(name,rows)=>`<table><thead><tr><th>${name}</th><th>Last
month</th><th>MTD</th><th>Last
7d</th><th>Yesterday</th><th>Now</th></tr></thead><tbody>`+
   rows.map(r=>`<tr><td>esc(r.n) < /td >< td >{nf(r.lm)}</td><td>nf(r.mtd) < /td >< td >
{nf(r.l7)}</td><td>nf(r.y) < /td >< tdclass = "
{r.wow>0?'up':'down'}">${pct(r.wow)}</td></tr>`).join('')+

`</tbody>${name==='Region'?`<tfoot><tr><td>TOTAL</td><td>${nf(D.total.last_month)}</td><t
d>nf(D.total.mtd) < /td >< td >{nf(D.total.last7)}</td><td>
nf(D.total.yesterday) < /td >< td >{pct(D.total.wow)}</td></tr></tfoot>`:''}</table>`;

$('period').innerHTML=tb('Service',D.services)+'<br>'+tb('Category',D.cats)+'<br>'+tb('Region',D
.regions);
}

function init(){
  ('lbl').textContent = D.label;('gen').textContent='Generated '+D.generated+' · data through
'+D.dates[D.dates.length-1];

('tabs').innerHTML = [['calendar','Calendar'],['trend','Trend'],['services','Services'],['categories','Categori
{t[0]}">${t[1]}</button>`).join('');

('ranges').innerHTML = [[7,'7d'],[15,'15d'],[30,'30d'],[60,'60d'],[90,'90d'],['all','All']].filter(r => r[0] ==
{r[0]}">${r[1]}</button>`).join('');
  const sv=[...new Set(D.series.map(s=>s.s))].sort(),rg=[...new Set(D.series.map(s=>s.r))].sort();

('fsvc').innerHTML = ' < optionvalue = "" > Allservices < /option > ' + sv.map(s => ` < option >
{esc(s)}</option>`).join('');
```

$('freg').innerHTML = '<option value="">Allregions</option>' + rg.map(s => `<option>{esc(s)}</option>`).join('');
  $('tabs').onclick=e=>{if(e.target.dataset.k){S.tab=e.target.dataset.k;render()}};
  $('ranges').onclick=e=>{const k=e.target.dataset.k;if(k){S.range=k==='all'?'all':+k;render()}};
  $('fsvc').onchange = e => S.svc = e.target.value; render();
('freg').onchange=e=>{S.reg=e.target.value;render()};
  if(S.range>D.dates.length)S.range='all';
  kpis();period();render();
}
init();
</script></body></html>
"""

def _row_json(r: Row) -> dict:
    return {"n": r.service, "lm": round(r.last_month, 2), "mtd": round(r.mtd, 2),
            "l7": round(r.last7, 2), "p7": round(r.prev7, 2), "y": round(r.yesterday, 2), "wow":
r.wow_pct}

def render_html(report: Report) -> str:
    t = report.total
    prev_month = report.run_date.replace(day=1) - timedelta(days=1)
    data = {
        "label": report.label,
        "unit": report.unit,
        "generated": datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M UTC"),
        "month": f"{report.run_date:%B %Y}",
        "cur_month": f"{report.run_date:%Y-%m}",
        "prev_month": f"{prev_month:%Y-%m}",
        "dates": [d.isoformat() for d in report.dates],
        "series": report.series,
        "forecast": None if report.forecast_month_end is None else round(t.mtd +
report.forecast_month_end, 2),
        "total": {"last_month": t.last_month, "mtd": t.mtd, "last7": t.last7,
                  "yesterday": t.yesterday, "wow": t.wow_pct},
        "services": [_row_json(r) for r in report.rows],
        "regions": [_row_json(r) for r in report.region_rows],
        "cats": [_row_json(r) for r in report.cat_rows],
        "anomalies": [{"s": a.service, "y": round(a.yesterday, 2), "b": round(a.baseline, 2)}
                      for a in report.anomalies[:8]],
        "inv": report.inventory,
        "inst": report.instances,

```
        "ec2t": report.ec2_types,
        "ec2days": report.ec2_days,
        "res": report.resources,
        "reskinds": report.res_kinds,
        "resdays": report.res_days,
    }
    blob = json.dumps(data, separators=(",", ":")).replace("</", "<\\/")
    return HTML_TEMPLATE.replace("__DATA__", blob)


def write_html(path: str, report: Report) -> None:
    tmp = path + ".tmp"
    with open(tmp, "w", encoding="utf-8") as fh:
        fh.write(render_html(report))
    os.replace(tmp, path)  # atomic: a web server never serves a half-written file
    log.info("HTML dashboard written to %s", path)


# ---------------------------------------------------------------------------
# Delivery
# ---------------------------------------------------------------------------
def post_json(url: str, payload: dict, name: str, retries: int = 3, timeout: int = 20) -> None:
    """POST JSON with retries. The URL is a secret and is never logged."""
    data = json.dumps(payload).encode("utf-8")
    last_err = "unknown error"
    for attempt in range(1, retries + 1):
        delay = 2 ** attempt
        req = urllib.request.Request(url, data=data, headers={"Content-Type": "application/json"})
        try:
            with urllib.request.urlopen(req, timeout=timeout) as resp:
                if 200 <= resp.status < 300:
                    log.info("%s notification sent (HTTP %s)", name, resp.status)
                    return
                last_err = f"HTTP {resp.status}"
        except urllib.error.HTTPError as exc:
            body = exc.read().decode("utf-8", "replace")[:200]
            last_err = f"HTTP {exc.code}: {body}"
            if exc.code < 500 and exc.code != 429:
                break
            ra = exc.headers.get("Retry-After") if exc.headers else None
            if ra and ra.strip().isdigit():
                delay = min(int(ra), 30)   # honour the server's back-off (Slack rate limit)
        except (urllib.error.URLError, TimeoutError) as exc:
            last_err = f"network error: {exc}"
```

```
        if attempt < retries:
            time.sleep(delay)
    raise RuntimeError(f"{name} delivery failed: {last_err}")


def write_csv(path: str, report: Report) -> None:
    with open(path, "w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["type", "name", "last_month", "month_to_date", "last_7_days",
"previous_7_days",
                    "yesterday", "wow_pct", "currency"])
        groups = [("service", report.rows), ("category", report.cat_rows),
                  ("region", report.region_rows), ("total", [report.total])]
        for kind, rows in groups:
            for r in rows:
                w.writerow([kind, r.service, f"{r.last_month:.2f}", f"{r.mtd:.2f}", f"{r.last7:.2f}",
                            f"{r.prev7:.2f}", f"{r.yesterday:.2f}",
                            "" if r.wow_pct is None else f"{r.wow_pct:.1f}", report.unit])
        w.writerow([])
        w.writerow(["daily_date", "total_cost", "currency"])
        for d, v in zip(report.dates, report.daily):
            w.writerow([d.isoformat(), f"{v:.2f}", report.unit])
        if report.ec2_types:
            w.writerow([])
            w.writerow(["ec2_region", "instance_type", f"cost_last_{report.ec2_days}d", "currency"])
            for e in report.ec2_types:
                w.writerow([e["r"], e["t"], f"{e['c']:.2f}", report.unit])
        if report.inventory:
            w.writerow([])
            w.writerow(["inventory_region", "running", "stopped", "other", "ebs_volumes", "ebs_gb",
                        "unattached_volumes", "unattached_gb", "elastic_ips", "idle_elastic_ips"])
            for i in report.inventory:
                w.writerow([i["r"], i["run"], i["stop"], i["oth"], i["vols"], i["gb"],
                            i["uvols"], i["ugb"], i["eip"], i["ieip"]])
        if report.resources:
            w.writerow([])
            w.writerow(["resource_id", "kind", "region", "name", f"cost_last_{report.res_days}d",
"currency"])
            for e in report.resources:
                w.writerow([e["id"], e["k"], e["r"], e["n"], f"{e['c']:.2f}", report.unit])
    log.info("CSV written to %s", path)


# ---------------------------------------------------------------------------
```

```
# Orchestration
# ---------------------------------------------------------------------------
def parse_args(argv=None) -> argparse.Namespace:
    p = argparse.ArgumentParser(
        description="AWS cost report: service / category / region wise, EC2 fleet, resource-level
cost, "
                    "calendar dashboard, rich terminal view, Slack/Teams.")
    p.add_argument("--profile", help="AWS profile name (default: standard credential chain)")
    p.add_argument("--label", default=os.getenv("COST_REPORT_LABEL", "AWS"),
help="Name shown in titles")
    p.add_argument("--top", type=int, default=12, help="Rows to list individually (default 12)")
    p.add_argument("--days", type=int, default=90, help="Daily history for calendar/graphs
(default 90, max ~400)")
    p.add_argument("--tag", help="Filter to a cost-allocation tag, e.g. Environment=production")
    p.add_argument("--include-credits", action="store_true", help="Include credits/refunds
(excluded by default)")
    p.add_argument("--anomaly-pct", type=float, default=30.0, help="Spike threshold vs 7-day
avg, percent")
    p.add_argument("--anomaly-min", type=float, default=5.0, help="Minimum spike in currency
units")
    p.add_argument("--no-ec2", dest="ec2", action="store_false", help="Skip EC2
cost-by-instance-type call")
    p.add_argument("--no-resources", dest="resources", action="store_false",
                   help="Skip resource-level cost call (needs resource-level data enabled in Cost
Explorer)")
    p.add_argument("--no-inventory", dest="inventory", action="store_false",
                   help="Skip the EC2/EBS/Elastic IP inventory scan of all regions")
    p.add_argument("--color", choices=("auto", "always", "never"), default="auto",
                   help="ANSI colours in the console view (default auto: on for a terminal)")
    p.add_argument("--html", metavar="PATH", help="Write the self-contained HTML dashboard
to PATH")
    p.add_argument("--dashboard-url", default=os.getenv("COST_DASHBOARD_URL"),
                   help="Public URL of the dashboard; adds an 'Open dashboard' link to
Slack/Teams")
    p.add_argument("--csv", help="Write tables + daily totals + EC2/resources to this CSV path")
    p.add_argument("--slack", action="store_true", help="Post to Slack (uses
SLACK_WEBHOOK_URL)")
    p.add_argument("--slack-webhook", metavar="URL", default=None,
                   help="Slack webhook URL (implies --slack). Visible in `ps`: prefer
SLACK_WEBHOOK_URL or --slack-webhook-ssm")
    p.add_argument("--slack-webhook-ssm", metavar="PARAM", default=None,
                   help="SSM Parameter Store SecureString holding the Slack webhook URL (or env
SLACK_WEBHOOK_SSM)")
    p.add_argument("--teams-webhook-ssm", metavar="PARAM", default=None,
```

```
            help="SSM Parameter Store SecureString holding the Teams webhook URL (or env
TEAMS_WEBHOOK_SSM)")
    p.add_argument("--ssm-region", default=None, help="Region of the SSM parameters
(default: session/AWS_REGION)")
    p.add_argument("--allow-empty", action="store_true",
            help="Do not fail when Cost Explorer returns no cost data (default: exit 1 instead of
posting a $0 report)")
    p.add_argument("--teams", action="store_true", help="Post to Teams (uses
TEAMS_WEBHOOK_URL)")
    p.add_argument("--dry-run", action="store_true", help="Print/write files only; never post to
Slack/Teams")
    p.add_argument("-v", "--verbose", action="store_true")
    return p.parse_args(argv)


def main(argv=None, ce_client=None) -> int:
    args = parse_args(argv)
    logging.basicConfig(level=logging.DEBUG if args.verbose else logging.INFO,
                format="%(asctime)s %(levelname)s %(message)s")
    if not 1 <= args.days <= 400:
        log.error("--days must be between 1 and 400")
        return 2

    slack_url = args.slack_webhook or os.getenv("SLACK_WEBHOOK_URL")
    slack_ssm = args.slack_webhook_ssm or os.getenv("SLACK_WEBHOOK_SSM")
    if args.slack_webhook:
        args.slack = True
    teams_url = os.getenv("TEAMS_WEBHOOK_URL")
    teams_ssm = args.teams_webhook_ssm or os.getenv("TEAMS_WEBHOOK_SSM")
    want_slack = args.slack or (not args.teams and not args.dry_run and bool(slack_url or
slack_ssm))
    want_teams = args.teams or (not args.slack and not args.dry_run and bool(teams_url or
teams_ssm))
    if args.dry_run:
        want_slack = want_teams = False
    if want_slack and not (slack_url or slack_ssm):
        log.error("--slack given but neither SLACK_WEBHOOK_URL nor --slack-webhook-ssm is
set")
        return 2
    if want_teams and not (teams_url or teams_ssm):
        log.error("--teams given but neither TEAMS_WEBHOOK_URL nor --teams-webhook-ssm
is set")
        return 2
```

```
    T.utf = "utf" in (getattr(sys.stdout, "encoding", "") or "").lower()
    if args.color == "always":
        T.color = True
    elif args.color == "never":
        T.color = False
    else:
        T.color = sys.stdout.isatty() and "NO_COLOR" not in os.environ and os.getenv("TERM") !=
"dumb"
    width = max(100, min(shutil.get_terminal_size((110, 30)).columns, 140))

    today = datetime.now(timezone.utc).date()
    try:
        session = None
        if ce_client is None:
            session = make_session(args.profile)
            ce = make_ce_client(session)
        else:
            ce = ce_client
        # Resolve webhook secrets first so a bad secret fails fast, before any paid Cost Explorer
call.
        if want_slack:
            slack_url = resolve_webhook(slack_url, slack_ssm, session, args.ssm_region, "Slack")
        if want_teams:
            teams_url = resolve_webhook(teams_url, teams_ssm, session, args.ssm_region,
"Teams")
        report = generate_report(ce, args.label, today, args, session)
    except Exception as exc:  # noqa: BLE001
        log.error("Could not build the cost report: %s", exc)
        return 1

    if not report.rows and not args.allow_empty:
        log.error("Cost Explorer returned no cost data (new account, wrong --profile/--tag filter, or
missing "
                  "ce:GetCostAndUsage permission). Nothing posted. Use --allow-empty to override.")
        return 1

    print(render_rich(report, args.top, width))

    if args.csv:
        write_csv(args.csv, report)
    if args.html:
        write_html(args.html, report)

    failures = 0
```

```
    if want_slack:
        try:
            post_json(slack_url, slack_payload(report, args.top, args.dashboard_url), "Slack")
        except RuntimeError as exc:
            log.error("%s", exc)
            failures += 1
    if want_teams:
        try:
            post_json(teams_url, teams_payload(report, args.top, args.dashboard_url), "Teams")
        except RuntimeError as exc:
            log.error("%s", exc)
            failures += 1
    return 1 if failures else 0


if __name__ == "__main__":
    sys.exit(main())
```